A Universal Cryptographic Sealing Primitive for Digital File Provenance in the Age of Generative AI

A Decentralized Trust Layer with a Verifiable Soft-Binding Resolution Registry for Content Provenance

POSITION & ARCHITECTURE PAPER · VERSION 2.0 — FINAL EDITION · JUNE 2026

Working / provisional system name. SEAL. The name is explicitly provisional: it collides with established projects — most notably Microsoft SEAL, the homomorphic-encryption library — and will be re-evaluated before public release. Nothing in the architecture depends on the name.

Paper type. A position and architecture (systematization) paper. It specifies a design, situates that design against the standards-of-record and the production transparency-log systems, states its guarantees as theorems with explicit assumptions and proof sketches, and commits to a falsifiable, adversarial evaluation plan (§9). Quantitative figures attributed to prior work are cited as such. The paper's own measurements — reference-contract gas, conformance vectors, client-side microbenchmarks (Appendices D–E) — are labeled with the environments that produced them and are explicitly *not* substitutes for the public-chain evaluation plan.

Statement of significance. This paper makes exactly three claims, each stated as a guarantee with explicit assumptions and proven (or honestly bounded) where indicated. It claims nothing else — in particular, it does not claim the hard/soft binding of what/who/when, which belongs to C2PA and JPEG Trust.

1. **A trustless anchor with interval timestamp semantics.** Tri-mode consensus is wrapped in a checkpoint hierarchy terminating in Bitcoin via OpenTimestamps, so anti-backdating holds even against the default mode's own sequencer — the bound is always cited at L1/Bitcoin level, never at L2 alone. Proven in Theorem 4 (Anti-backdating — G4, R5), §6.1, with the architecture in §4.5 (Fig. 4).
2. **A verifiable soft-binding resolution registry.** The decentralized analogue of C2PA's Soft Binding Resolution API, rebuilt in the Certificate-Transparency pattern — on-chain append-only log as sole source of truth, versioned deterministic index rule, permissionless off-chain indexers, anchored index roots, proof-carrying results — so positive matches are trust-minimized while completeness is priced honestly as a k-of-n indexer quorum. Soundness proven in Theorem 5 (Resolution soundness — G5, R15), §6.1; the quorum assumption is stated and bounded in §6.2; the construction is §4.4 (Fig. 3) and Appendix B.
3. **Longevity engineering with confidential sealing.** Salted commitments make the on-chain record leak nothing about sealed content — defeating dictionary and confirmation oracles — while chunked Merkle commitments support selective disclosure, and crypto-agile epochal renewal keeps "anchored before the break" provable across cryptographic generations, with exportable self-contained verification bundles that outlive every operator. Confidentiality proven in Theorem 2 (Hiding — G2, R9) with the §4.6 operational rules; renewal preservation proven in Theorem 6 (G6, R17); the mechanisms are §4.6, §4.8, §4.9, and §4.13.

Executive Summary

The problem. Generative AI made convincing forgery free, and decades of format migration made bit-exact hashes brittle. The two failures compound in opposite directions: a file that was merely re-encoded looks tampered, while a file that was expertly forged looks authentic. The trustworthy question is therefore no longer "does it look genuine?" — appearance has stopped being evidence — but "is this exactly what was registered, by which key, and when?" That question is answerable. It is a question of provenance, not truth.

What exists. Industry has already standardized the binding of what/who/when: C2PA / Content Credentials and JPEG Trust (ISO/IEC 21617-1:2025) attach signed manifests carrying a cryptographic hash (hard binding) and a perceptual fingerprint or watermark (soft binding). SEAL does not compete with that binding. It addresses what the standards leave open. Their trust roots are centralized — manifest repositories and CA trust lists. Recovering provenance for a stripped or transcoded file is delegated to centralized lookup services. There is no public anti-backdating timestamp without a trusted authority. And nothing guarantees that verification keeps working after the infrastructure — or the cryptography — ages out.

What SEAL is. A single sealing primitive for any file type. Sealing computes a hard hash plus per-modality fingerprints, signs a structured commitment client-side, anchors it on a public ledger wrapped by checkpoints that terminate in Bitcoin, and pins the bytes (optionally encrypted) to content-addressed storage. Verification is free, runs client-side, and outputs an auditable verdict vector under a named policy. Recovery without an identifier — the soft-binding resolution problem — is served by a verifiable index: the chain holds the append-only log and periodic index roots; anyone can rebuild the index from public data by a deterministic rule; and every

answer carries inclusion proofs, so a positive match never requires trusting the service that returned it. Confidential mode seals secrets as salted commitments — nothing about the content is learnable from the on-chain commitment, by dictionary attack or otherwise (Theorem 2, with the §4.6 operational rules for the auxiliary fields) — and supports releasing provably-authentic fragments later. Epochal renewal re-anchors all seals under fresh algorithms, so a seal made under today's cryptography remains provably pre-quantum decades from now. And every seal exports a self-contained verification bundle that still verifies offline after any operator, indexer, or even the L2 itself disappears.

What SEAL is not. It does not detect deepfakes; instead it inverts the burden of proof, so that unsealed content becomes the suspect class. It does not prove authorship or ownership: a seal is an attestation of possession, by design, with multi-attestation allowed. And it does not pretend that similarity-search completeness can be proven today — positive results carry proofs, while "no match" rests on an independent-indexer quorum that is stated, bounded, and measured in the evaluation plan.

Status. Specification with executed artifacts. Phase 1 — the bit-exact spine — is buildable today from standard primitives: a reference implementation has produced executed conformance vectors (real hashes, signatures, guids; tamper, replay, and dictionary attacks demonstrated and defeated — Appendix D), a reference contract passing its full in-EVM test suite with measured gas, and client-side measurements including real post-quantum dual-signatures (Appendix E). Phase 2 — fingerprints plus verifiable resolution — is the research program, with pass/fail experiments E1–E12 including poisoning, dictionary, and infrastructure-death drills.

Abstract

Generative AI made convincing forgery of digital content trivial, while longitudinal preservation made the bit-exact integrity assumption fragile: any lawful transcoding breaks a classical hash with no malicious change. The standards-of-record — C2PA / Content Credentials and JPEG Trust (ISO/IEC 21617-1:2025) — already bind what / who / when into signed manifests via hard bindings (cryptographic hashes) and soft bindings (perceptual fingerprints / watermarks). They leave four gaps: trust anchored in centralized repositories and CA trust lists; soft-binding resolution — recovering provenance for a stripped or transcoded artifact, with no identifier — externalized to centralized services; no public anti-backdating timestamp without a trusted authority; and no longevity engineering, so verification dies with the infrastructure. SEAL addresses exactly these. Its anchor is a tri-mode ledger wrapped in a checkpoint hierarchy terminating in Bitcoin, yielding interval timestamp semantics that survive even a malicious L2 sequencer. Its resolution registry is a verifiable index in the Certificate-Transparency pattern: an on-chain append-only log is the sole source of truth, a versioned deterministic rule lets anyone reconstruct the similarity index off-chain, index roots are anchored on-chain, and every query result carries inclusion proofs — positive results are trust-minimized, and completeness is honestly priced as an independent-indexer quorum assumption. A single file-type-agnostic Seal record unifies client attestation (possession, not origination), a coded revocation lifecycle, multi-attestation semantics, per-modality fingerprint sets, confidential (salted) sealing with selective disclosure, and crypto-agile epochal renewal for decade-to-century horizons. The paper provides a formal model with six stated guarantees and their reductions (§3.3, §6.1), a reference parameterization and minimal contract interface backed by a tested reference implementation with measured gas (§4.14–§4.15, Appendix E), a worked cost model (§7), falsifiable adversarial evaluation (§9), and executed conformance vectors including real post-quantum dual-signatures (Appendices D–E). The result is a decentralized, abuse-resistant, legally bridgeable source of provenance: an artifact is exactly what was registered, attested by which key, and when — recoverable and verifiable by anyone, indefinitely, without a trusted third party.

Keywords. Content Provenance · C2PA / Content Credentials · JPEG Trust · Soft-Binding Resolution · Transparency Logs · Distributed Ledger · Perceptual Hashing · Decentralized Timestamping · Crypto-Agility · Post-Quantum · IPFS.

Contents

Executive Summary · Abstract

1. **Introduction** — the two broken axes; the four gaps; contributions; motivating applications
2. **Background, Standards, and Related Work** — standards-of-record; seven synthesized systems; transparency logs ("why not X"); capability matrix; intellectual lineage (§2.5); the Nakamoto–Buterin design inheritance (§2.6, Fig. 1); the 2026 frontier survey (§2.7); the conservative-core principle from the cryptographic failure record (§2.8)
3. **Threat Model, Requirements, and Formal Definitions** — adversaries (a)–(q); R1–R20; the scheme Σ and guarantees G1–G6; hypotheses & methodology (§3.4)
4. **Architecture** — §4.1 layers (Fig. 2); §4.2 fingerprint dispatch; §4.3 Seal record & commitment; §4.4 verifiable resolution index (Fig. 3); §4.5 anchoring & governance (Fig. 4); §4.6 confidential sealing (Fig. 5); §4.7 lifecycle; §4.8 renewal; §4.9 verification bundles; §4.10 verification UX & policies (Fig. 6); §4.11 identity overlays; §4.12 standards bridges; §4.13 selective disclosure; §4.14 parameterization; §4.15 contract interface; §4.16 scale & capacity; §4.17 frontier integrations (Fig. 7)
5. **Protocol Flows** — Seal / Verify / Resolve / Revoke / Commit-Reveal / Renew
6. **Security Analysis** — Theorems 1–6 with proof sketches; limitations stated precisely; comparative failure matrix (§6.3)
7. **Cost and Scalability** — worked model; century-scale endowment economics (§7.1)
8. **Implementation Pathway** — three additive phases
9. **Evaluation Plan** — E1–E12 incl. adversarial experiments; red team
10. **Discussion** — privacy; legal admissibility (FRE 902, Daubert, eIDAS, MLETR); OAIS archival integration; standards pathway; deepfakes; operations; sustainability; what SEAL does not solve
11. **Conclusion, Open Problems, and the Five-Year Research Agenda**

References [1]–[65] · Appendix A (verification bundle schema) · Appendix B (index rule v1 + dispute procedure) · Appendix C (notation, glossary & acronyms) · Appendix D (worked example & executed conformance vectors) · Appendix E (reference microbenchmarks, measured) · Appendix F (anticipated objections) · Appendix G (deployment case studies with failure analysis)

How to read. Practitioners building on the design should read §4, §5, and Appendix D; theorists should read §3 and §6; decision-makers should read the Executive Summary, §2.8, §6.3, and §7.

Part I — The Problem and the Landscape

1 Introduction

The generative-AI era has broken the economics of documentary trust along two independent axes at once. For most of the history of records, forgery was expensive: reproducing an official document convincingly required skill, equipment, and access, and a bit-exact cryptographic hash was, for digital artifacts, a sufficient integrity guarantee. That settlement — costly forgery on one side, cheap and reliable integrity checking on the other — held for decades. It no longer holds, and the two failures are not the same failure:

- **Forgery of content is now trivial.** Generative models synthesize documents, images, audio, and video indistinguishable from authentic artifacts. The visual and forensic plausibility of a file has ceased to carry evidentiary weight: "it looks genuine" is no longer evidence of anything. The burden of trust consequently shifts from the *appearance* of an artifact to its *provenance* — from what the file looks like to where it verifiably came from.
- **Bit-exact integrity is brittle under preservation.** Artifacts are routinely transcoded, recompressed, and migrated across decades as formats die and storage systems are replaced. A binary hash breaks on the first lawful re-encoding, which means that under real archival conditions a hash mismatch carries no signal: malicious alteration becomes indistinguishable from legitimate format shift.

Neither extreme suffices, and the architecture this paper presents is built on refusing to choose between them. A purely binary-hash system cannot survive the routine life of a file; a purely perceptual system lacks cryptographic rigor. The achievable — and necessary — guarantee is *provenance, not truth*: not that a document is true, but that it is the one that was registered, attested by which key, and when — and that this remains checkable by anyone, indefinitely. Everything that follows is an engineering of that sentence.

1.1 The gaps the standards-of-record leave open

Binding *what / who / when* into a signed manifest is not an open problem, and this paper does not pretend otherwise. C2PA / Content Credentials and JPEG Trust (ISO/IEC 21617-1:2025) solved and standardized that binding, with hard bindings (asset hashes) and soft bindings (perceptual fingerprints / watermarks) signed under X.509 and validated against a trust list. Re-claiming that binding would be indefensible. Four genuine gaps remain, and they define the territory this design occupies:

- **Centralized trust roots.** C2PA anchors authenticity in manifest repositories and certificate-authority trust lists — institutional PKI, exactly the form of centralization that AI-era skepticism strains. A provenance regime whose root of trust is a curated list of authorities inherits every governance, capture, and jurisdiction problem of that list.
- **Externalized — and unverifiable — resolution.** When a manifest is stripped (a screenshot, a metadata-dropping re-upload) or a file is transcoded, recovering provenance requires a Soft Binding Resolution lookup, which C2PA delegates to centralized services. No public registry exists that, given only a transcoded artifact and no identifier, returns its provenance. Hosting an index "on a blockchain" is not the answer either: resolution must be *verifiable*, meaning any answer can be checked against the trust anchor, and anyone can rebuild the index from public data (§4.4).

- **No anti-backdating public timestamp without a trusted authority, and no unified lifecycle.** Existence proofs, revocation, and file-type universality are scattered across separate systems; no single primitive carries an artifact from sealing through revocation, renewal, and decades-later verification.
- **No longevity engineering.** The standards assume living infrastructure — repositories reachable, CAs solvent, resolution services maintained. Nothing guarantees that a manifest verifies after its infrastructure dies, that signatures survive cryptanalytic progress, or that proofs outlive the ledger they were anchored to. For archival horizons this is the dominant failure mode, and the architecture treats it as such (§4.5, §4.8, §4.9).

1.2 Contributions

We explicitly do not claim the hard/soft binding of what/who/when — that is C2PA/JPEG Trust, and the credit stays with them. The contributions of this work are:

- **A trustless anchor with interval timestamp semantics.** Tri-mode consensus (public L2 default; permissioned consortium; public-read/permissioned-write), wrapped in a checkpoint hierarchy terminating in Bitcoin via OpenTimestamps, so that anti-backdating holds even against the default mode's own sequencer (§4.5, Fig. 4).
- **A verifiable soft-binding resolution registry.** The decentralized analogue of C2PA's Soft Binding Resolution API, rebuilt in the Certificate-Transparency pattern: an on-chain append-only log as sole source of truth; a versioned deterministic index rule; permissionless off-chain indexers; anchored index roots; proof-carrying results. Positive matches are trust-minimized; completeness is priced honestly as a k-of-n indexer quorum (§4.4, Fig. 3, Appendix B).
- **A single file-type-agnostic primitive** unifying client attestation (possession, not origination), a coded revocation lifecycle, multi-attestation semantics, per-modality fingerprint sets, on-chain lineage, and format-aware dispatch — mapped one-to-one onto C2PA vocabulary (§4.2, §4.3, §4.7, §4.12).
- **Confidential sealing and selective disclosure.** Salted commitments neutralize dictionary/confirmation oracles against sealed-but-secret content; chunked Merkle commitments let an archive release page 3 and prove it belongs to the sealed whole (§4.6, §4.13). This also supplies the GDPR practical-erasure mechanism.
- **Longevity engineering.** Crypto-agility via algorithm identifiers, ERS-style epochal renewal proving "anchored before the break," an optional ML-DSA dual-signing archival profile, exportable self-contained verification bundles, and a registry migration protocol (§4.8, §4.9).
- **A formal model with an adversarial, falsifiable evaluation.** The scheme Σ and guarantees G1–G6 are defined in §3.3; Theorems 1–6 with reductions appear in §6.1; the adversary catalogue (a)–(q) includes index poisoning, malicious verifier UIs, sequencer manipulation, and future cryptanalysis; experiments E1–E12 include poisoning, dictionary, sunset-survival, and indexer-consistency drills (§9).

1.3 Motivating applications

Three applications, drawn from institutional practice, exercise nearly every requirement the design must satisfy:

- **Embargoed public inquiries.** Documentary evidence for inquiries can remain closed for the best part of a century. Sealing at deposition lets the public verify, decades later, that releases are unaltered and were attested at deposition. This single use case motivates anti-backdating (R5), reproducibility (R10, R16), confi-

dential sealing — the index must not become a confirmation oracle against embargoed content (§4.6) — selective disclosure for partial release (§4.13), and renewal across cryptographic generations (R17).

- **Reproducible computational research.** A university seals a dataset (via DOI) together with the hash of the code that derives the result. Years later, integrity challenges are answered by evidence that data and code match deposition — the clearest justification for `modelHash`-style pinning.
- **Journalism and capture-to-publication custody.** Newsroom or citizen footage is sealed at ingest; platform re-encodes strip manifests; the resolution registry recovers the original seal from the stripped derivative, and the lineage chain (`prevGuid`) records each edit (§4.4, §4.7).

> **Section takeaways.** Generative AI breaks documentary trust along two independent axes — trivial content forgery and brittle bit-exact integrity — and the achievable guarantee is provenance, not truth. The signed what/who/when binding is solved by C2PA/JPEG Trust; the four open gaps are centralized trust roots, unverifiable no-identifier resolution, the absence of an authority-free anti-backdating timestamp with a unified lifecycle, and the absence of longevity engineering. SEAL's contributions target exactly those four gaps.

2 Background, Standards, and Related Work

2.1 The standards and frontier landscape

The design positions itself against the standards-of-record deliberately and precisely: each item below states what the incumbent does well, and only then what SEAL adds.

- **C2PA / Content Credentials.** Signed manifests (assertions + claim + claim signature) in a Manifest Store, X.509-signed, validated against the C2PA Trust List; offline-verifiable; deployed across Adobe, Google, OpenAI, Meta, Sony, Leica, Samsung; endorsed by U.S. NSA/CISA. Stronger than SEAL where it is strong: standardized, shipping, hardware-attested capture. What SEAL adds: a decentralized anchor, verifiable resolution, and longevity engineering.
- **Durable Content Credentials (C2PA 2.1+).** Hard binding + soft binding (Digimarc-class watermarks, fingerprint lookups) + a Soft Binding Resolution API. SEAL's `exactHash` / fingerprint set is precisely this pairing; SEAL's contribution is the verifiable, decentralized resolution registry, not the pairing itself.
- **JPEG Trust — ISO/IEC 21617-1:2025.** Elevates provenance to an International Standard (Trust Records, Trust Manifests, Trust Profiles → Trust Reports). SEAL's verification policy profiles (§4.10) deliberately mirror Trust Profiles.
- **Perceptual-hash-on-blockchain (closest academic competitor, 2026).** pHash + chain anchor + Merkle batching + an MPT/BK-tree searchable registry — for images, verification-only. SEAL adopts its index structures as one shard family but relocates them into the verifiable-index pattern (§4.4), and adds attestation, lifecycle, universality, and longevity.
- **Watermarking (SynthID, Digimarc).** In-artifact soft bindings, complementary to SEAL's external seal; a concrete fingerprint family for the dispatch table (§4.2).
- **Regulatory drivers.** EU AI Act Art. 50 mandates machine-readable marking of synthetic media; NSA/CISA endorse Content Credentials; eIDAS provides qualified timestamps with statutory evidentiary weight (§4.12). Provenance infrastructure is becoming compliance, not research.

2.2 The seven synthesized academic systems (summary)

Seven prior academic systems are synthesized into the design, and the credit for each inherited mechanism stays explicit — that traceability is part of the architecture's strength, not a liability. Compressed to their contributions:

- **ARCHANGEL [1]:** content-aware DNN hashing for scanned archives, model-artifact archival, DROID format dispatch, cross-institution permissioned consensus, verifier-pays sustainability.
- **Meher et al. [2]:** a requirements checklist — oracles, automation, the resilience triad, monitoring, APIs, SSI, compliance.
- **Tio et al. [3]:** engineering-grade notarization — factory + event-driven storage, client-side ECDSA, revocation, gas/latency benchmarks with honesty caveats, meta-transactions, and the ECDSA/EdDSA trade-off analysis.
- **Tejaswini et al. [4]:** issuer-side write RBAC, domain `documentId`, and the substrate-agnostic spine evidence.
- **Ansari & Zade [5]:** Merkle batch timestamping, TSA elimination, contribution-weighted attribution polling, consent/version control.
- **Chinnasamy et al. [6]:** read-side RBAC, on-chain access audit, ML insider-threat forecasting, parallel verification.
- **Birhade et al. [7]:** hosted-IPFS pragmatics, QR/link verification UX (hardened in §4.10), and an independent gas cross-check.

2.3 What none of the above answer: transparency logs

The systems closest to SEAL's actual contribution are not academic blockchain-notarization papers but production append-only-log infrastructures — the quiet workhorses of internet trust. A credible design must therefore answer the hardest fair question first: *why a blockchain at all?* The table states, for each candidate infrastructure, what it provides, why it alone cannot fill SEAL's slot, and what role it nonetheless plays inside the design.

System	What it provides	Why it alone is insufficient here	Role in the design
RFC 3161 TSA	Signed timestamps; legal acceptance	A trusted authority; retroactive risk on TSA key compromise; no public log	Optional eIDAS dual-stamp (§4.12)
KSI (Guardtime)	Hash-calendar timestamping without PKI, national scale (Estonia)	Operator-run calendar; permissioned core	Pattern source for renewal chains (§4.8)
Certificate Transparency (RFC 6962)	Append-only verifiable logs, inclusion + consistency proofs, gossip audit — at world scale	Scoped to certificates; logs are operated services; no similarity search; no native existence currency	The architectural pattern for the resolution registry (§4.4)
Sigstore / Rekor	Free production transparency log for artifact signing	Centralized log operator; no media fingerprints; no resolution	Phase-1 interoperability target; pattern source
OpenTimestamps	Free, aggregated Bitcoin-anchored existence proofs	Existence only — no attestation, lifecycle, fingerprints, or resolution	Terminal anchor of the checkpoint hierarchy (§4.5)

System	What it provides	Why it alone is insufficient here	Role in the design
Arweave / Filecoin	Persistent storage economics	Storage is not attestation or resolution	Optional pinning backend (R14)
Public L2 + events	Cheap programmable anchoring; permissionless writes; composability	Sequencer trust window; substrate mortality	Default anchor — wrapped by checkpoints and bundles (§4.5, §4.9)

The answer this design gives is structural. SEAL needs three properties no transparency log supplies together: *permissionless writes* — anyone can seal, whereas CT logs and Rekor gate writes through operators; *programmable lifecycle* — revocation, renewal, and fees, whereas logs are passive; and *operator-independent survival* — bundles plus Bitcoin checkpoints, whereas a log's proofs die with its operator. The architecture therefore uses a chain where a chain is necessary and the CT pattern where computation must scale — and it inherits the legal bridge from RFC 3161/eIDAS rather than fighting it.

2.4 Capability matrix and the gap, restated

The capability matrix places the standards-of-record, the closest academic competitor, the seven synthesized systems, and the transparency-log infrastructures side by side. Each cell records what the system actually delivers, including the qualifications.

Capability	C2PA / JPEG Trust	Mohit et al. 2026 [8]	Seven systems (best of [1]–[7])	CT / Rekor [27][28]	OpenTimestamps [24]	SEAL
Hard binding (bit-exact)	Yes	hash	Yes	Yes (artifacts)	Yes	Yes, algorithm-agile
Soft binding (content-aware)	Yes	pHash (images)	DNN (scans) [1]	—	—	Per-modality fingerprint sets (R16)
No-ID resolution	Centralized API	Yes, image-only, unverifiable hosting	—	—	—	Verifiable index: proofs on every answer
Client attestation + lifecycle	X.509; manifest-level	—	ECDSA + revoke [3]	Signing identities (Fulcio)	—	EIP-712 + coded revocation, multi-attestation
Decentralized trust anchor	No (CA trust list)	Yes	Partial	Operated logs	Bitcoin	Tri-mode + checkpoints to Bitcoin
Anti-backdating w/o TSA	—	Block time	Block time	Log time (operator)	Yes	Interval semantics ⟨L2, L1, BTC⟩ (R19)
Confidential sealing	Manifest redaction	—	—	—	Hash-only (private by default)	Salted commitments + selective disclosure
Longevity (renewal, offline proofs)	Repository-dependent	—	—	Log-dependent	Bitcoin-dependent (good)	Epochal renewal + exportable bundles (R17, R20)

The gap, restated. Neither the centralized standards-of-record (no decentralization, externalized resolution, no longevity engineering), nor the academic systems (no resolution, fragmented capabilities), nor the transparency logs (no permissionless writes, no lifecycle, no similarity resolution) provide the bottom-right column: a decentralized trust anchor with a verifiable no-identifier resolution registry, unified attestation/lifecycle/universality, confidential sealing, and century-scale verification. That combination — and only that combination — is the gap SEAL occupies.

2.5 Intellectual lineage: six decades of authentication research

SEAL is not a 2020s improvisation; every component descends from an identifiable research lineage, and locating the design in that lineage is what separates an architecture from an arrangement of fashionable parts. Five literatures converge here:

- **Diplomatics and archival science.** The analysis of documentary authenticity predates computing by centuries; its modern digital form — Duranti's archival diplomatics and the InterPARES projects [51] — established that a record's trustworthiness decomposes into *identity* and *integrity* maintained across custody. SEAL's Seal record is precisely a machine-verifiable diplomatic envelope: identity (signer, issuer, `documentId`), integrity (`exactHash`, fingerprints), and custody (`prevGuid` lineage, renewal chain). The OAIS reference model (ISO 14721) and the Trusted Digital Repository audit standards (ISO 16363) [50] define where the primitive operates institutionally (§10).
- **Cryptographic timestamping.** Haber & Stornetta's 1991 "How to Time-Stamp a Digital Document" [40] founded the field — and, via linked hashing, is the direct ancestor of blockchains; Bayer–Haber–Stornetta added Merkle aggregation in 1993 [41], and Buldas et al. formalized binding linking schemes [42]. SEAL's anchor + batching + OpenTimestamps hierarchy is this exact line of work, industrialized; the consortium mode echoes the original surety model.
- **Authenticated data structures and tamper-evident logs.** Tamassia's ADS framework [43] — answer-with-proof query structures — and Crosby–Wallach's tamper-evident logging [44] supply the theory the verifiable index (§4.4) instantiates; Certificate Transparency [27][46] and key transparency (CONIKS [45], later deployed by major messengers) proved the pattern at planetary scale. SEAL's contribution is extending the pattern from exact-match dictionaries to similarity search — which is exactly why completeness becomes the open problem (§11).
- **Perceptual hashing and its adversarial literature.** From Zauner's benchmarking of perceptual image hashes [47] through industrial deployments (PhotoDNA, PDQ/TMK [25]) to the modern attack literature — Struppek et al. break NeuralHash with both evasion and forced collisions [48] — the field's consistent lesson is that perceptual hashes are robust against benign transforms and weak against adaptive adversaries. SEAL encodes that lesson as architecture rather than disclaiming it in a footnote: R3 is scoped to non-adversarial transcodes, E3 measures the attack surface, and policy profiles make the operating point explicit rather than implied.
- **Watermarking impossibility.** Recent theory shows strong watermarking of generative-model output is impossible against motivated attackers under natural assumptions [49]. This result is load-bearing for SEAL's thesis: if marks inside the artifact cannot survive adversaries, provenance must live *outside* the artifact — in registries — and watermarks demote to best-effort soft bindings. The inversion defense (§10) is the architectural consequence.

> **Section takeaways.** The design synthesizes five mature literatures — diplomatics, timestamping, authenticated data structures, perceptual hashing, and watermarking theory — and its two honest weaknesses (similarity-search completeness, adversarial fingerprint robustness) are exactly the two places where those literatures themselves are open.

2.6 The Nakamoto–Buterin design inheritance

Two founding texts of the field state, between them, almost the entire design philosophy this paper executes — and the debt should be explicit rather than ambient (Figure 1).

- **Nakamoto 2008: proof instead of trust, and the chain as a timestamp server.** The Bitcoin paper's problem statement — "what is needed is an electronic payment system based on cryptographic proof instead of trust" — and its solution's very first move — "the solution we propose begins with a timestamp server" — are, read carefully, a provenance design before they are a payments design [17]: hash content, publish the hash into an ongoing chain, and existence-at-time becomes publicly checkable with no notary. SEAL's R5/R6 are this exact program applied to files rather than transactions; the persistence assumption on which Theorem 4 rests is the Nakamoto consensus property later formalized by Garay–Kiayias–Leonardos [38]; and the checkpoint hierarchy's terminal anchor (§4.5) is, fittingly, Nakamoto's own chain. A second, quieter inheritance: Bitcoin's validation core is famously small and frozen — the same instinct behind SEAL's one-page, admin-key-free registry ABI (§4.15).
- **Buterin 2013–2020: programmable trust, under credible neutrality.** Ethereum's contribution [18] is the part a passive timestamp chain cannot do: *lifecycle as code*. SEAL's coded revocation (§4.7), epochal renewal (§4.8), sealing fees and rate limits (R18), and commit-reveal (§5.5) are smart-contract logic in precisely Buterin's sense — rules that execute rather than operators who decide. The governing constraint comes from Buterin's later principle of *credible neutrality* [54]: a mechanism whose fairness is visible in its design — it names no privileged parties, its execution is open and verifiable, it stays simple, and it leaves minimal discretion. Every contested choice in SEAL resolves toward that principle: permissionless writes and checkpointing, a deterministic public index rule instead of an operator's database (§4.4), takedown at the accountable policy layer but never on-chain (R18), immutable registries with no upgrade keys, and verification bundles that are, in effect, light-client verification in the sense Buterin's trust-model taxonomy ranks most robust [55] — trust mathematics and widely replicated headers, not servers.

The inheritance is structural, not ideological: SEAL takes the founders' mechanisms while rejecting maximalism. Modes B/C keep institutions first-class (issuer identity, consortium governance); eIDAS dual-stamping embraces the legal system rather than routing around it (§4.12); and the design happily concedes that identity vetting and entitlement are social problems solved by overlays, not consensus (§4.11). What the founders supply is the part institutions cannot: a record nobody — including its operators — can quietly rewrite.

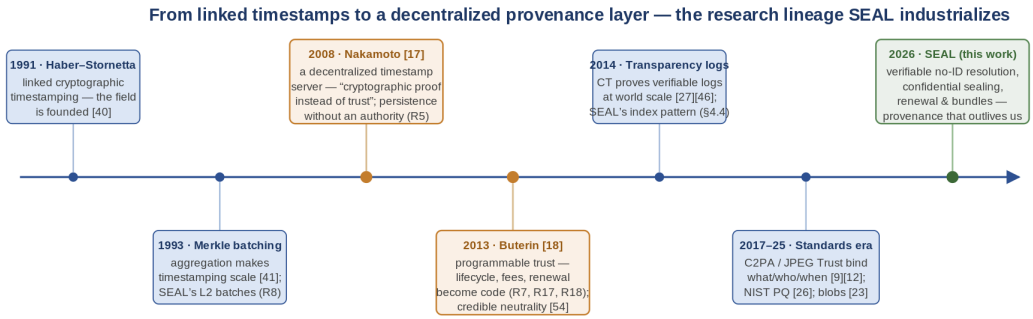


Figure 1. The lineage SEAL industrializes: each hop keeps its predecessor's guarantee and removes one dependency — on a notary (1991→2008), on a fixed function (2008→2013), on an operator (2013→2014), on living infrastructure (→2026).

2.7 The 2026 frontier: where the stack is going, and why it converges on this design

A snapshot of the frontier as of June 2026, drawn from current ecosystem reporting. The pattern across all seven fronts is the same: every major trend either lowers SEAL's costs, shrinks one of its stated trust exposures, or supplies an optional layer the architecture already reserved a slot for.

- **The regulatory clock has nearly run out.** EU AI Act Article 50 — machine-readable marking and detectability of synthetic content — applies from 2 August 2026, with a transition for pre-existing generative systems to 2 December 2026 and a supporting Code of Practice finalized mid-2026 [14]. Provenance infrastructure stops being optional this year; widely cited industry projections meanwhile put deepfake files shared online at roughly half a million (2023) and over eight million (2025). The window in which "unsealed is normal" closes — exactly the inversion thesis of §10.
- **Hard bindings are now born in hardware.** C2PA capture has crossed from specification to shipping silicon: Leica's M11-P signs every frame via a dedicated security chip; Sony ships C2PA across its α flagships and the PXW-Z300 — the first natively-signing camcorder; Google's Pixel 10 signs every photo by default with hardware-backed keys and an on-device timestamping authority; Samsung credentials AI-edited output. For SEAL this upgrades L0/L1 inputs: capture-time attestation feeds the identity overlays of §4.11, and the sealing pipeline inherits a hardware root it does not have to build — narrowing the identity gap of §4.11.
- **Anchoring costs are collapsing on schedule.** Ethereum's Fusaka upgrade (live 3 December 2025) activated PeerDAS data-availability sampling [60], with blob capacity stepped to a 14-target/21-max regime through 2026 and L2 fee reductions of 40–60% reported immediately, trending toward an order of magnitude. The §7 cost model was deliberately built on mechanisms, not prices — every one of these steps moves R8 further inside its envelope. Based rollups and preconfirmations, now maturing, sequence L2 transactions at L1: a future Mode-A variant on a based rollup shrinks adversary (n)'s window toward L1 inclusion itself, weakening the one trust concession §4.5 had to make.
- **Verifiable computation became an engineering line-item.** zkVMs crossed the real-time threshold in 2025–26 [61]: SP1-class systems prove the overwhelming majority of Ethereum L1 blocks in under 12 seconds on 16 commodity GPUs, with average proof costs measured in cents and falling an order of magnitude year-over-year. Consequence for SEAL: running the deterministic index rule (Appendix B) inside a zkVM and anchoring a validity proof with each `IndexCheckpoint` is no longer a research fantasy but a costed engineering project — a concrete path on open problem 1 (§11), upgrading build-correctness from quorum-checked to proof-carried. The honest remainder: succinct completeness for similarity queries still needs index designs that are proof-friendly.

- **Economic security became rentable.** Restaking matured into live infrastructure [59] — slashing on Ethereum mainnet since April 2025, redistributable slashing, data-availability and oracle services in production, tens of billions in restaked collateral. SEAL's indexer quorum is an unusually good fit for an actively-validated service: a divergent index root is an objectively attributable fault decidable from public data (Appendix B.3) — the cleanest slashing condition that exists. §4.17 specifies the integration; §2.8 specifies its hard boundary.
- **Personhood and web-origin proofs went mainstream.** World ID 4.0 (April 2026) [62] reports ~18 million Orb-verified humans and 450+ million verifications, with unlinkable per-relying-party proofs and integrations across mainstream platforms; zkTLS systems carry Web2 facts into verifiable contexts without revealing documents. For SEAL these are reserved-slot options: per-human rate-limiting for R18 without identity disclosure, Sybil-resistant attribution polling for L8, and zkTLS-attested web-origin evidence entering the metadata of a seal at capture (§4.17).
- **The entitlement layer is being built by others — as designed.** Programmable-IP chains (Story-class [64]: on-chain licensing terms, attribution graphs, proof-of-creativity) and shipping C2PA+chain hybrids (Numbers-class [65]) populate exactly the layer SEAL deliberately externalizes (§6.2): ownership, licensing, monetization. SEAL's `prevGuid` lineage is the natural substrate for their derivative graphs; composition, not competition (§4.17).

> **Section takeaways.** Four forces — regulation with a 2026 deadline, capture hardware, collapsing anchor costs, and commodity verifiable compute — are converging on provenance infrastructure. None of them occupies SEAL's slot (verifiable resolution + longevity engineering), and each one strengthens it: the frontier is building SEAL's optional layers and lowering its bills.

2.8 Lessons from cryptographic failure: the conservative-core principle

A design that asks institutions to trust it for a century owes them an answer to one question: *what happened when systems like this failed before?* The cryptographic failure record is not an embarrassment to be cited in passing; it is the most informative dataset the field possesses, and three canonical failures shape SEAL's deepest design rule. Each is worth examining in full, because each failed in a different place — the mathematics, the implementation, and the most-reviewed code in existence.

- **Zcash / BCTV14 (2018–2019) — the undetectable counterfeit.** A subtle soundness flaw in the BCTV14 zk-SNARK construction — extra elements mistakenly emitted by the trusted-setup procedure — allowed a cheating prover to transform a proof of one statement into a valid-looking proof of another: unlimited, undetectable counterfeiting of shielded value. The flaw survived years of expert review before Ariel Gabizon found it in March 2018; remediation required a network-wide migration (Sapling) and eleven months of secrecy [56]. Three lessons follow. Novel cryptographic complexity is attack surface. Trusted setups are standing liabilities. And the worst failure mode is the one that *cannot be observed*: because shielded value is hidden by design, no one can say to this day whether counterfeiting occurred.
- **"Frozen Heart" (2022) — correct math, broken code.** Trail of Bits disclosed a family of Fiat–Shamir implementation vulnerabilities across multiple proof systems (Girault, Bulletproofs, PlonK implementations): the papers were sound, the transcripts hashed by the code were not, and forgeries followed [57]. The lesson generalizes beyond zero-knowledge: even when the mathematics is right, every implementation is a *separate* soundness claim, and a security argument that stops at the paper has proven nothing about the deployment.

- **Bitcoin CVE-2018-17144 — the minimal core also fails.** An inflation-permitting bug shipped in the most conservative, most reviewed codebase in the field; it was caught because independent eyes and implementations existed [58]. The lesson: minimalism reduces failure probability but does not eliminate it; monoculture is the risk multiplier, and recomputability plus implementation diversity are the antidote.

The conservative-core principle. SEAL's answer — and this paper's deepest commitment — is that *the trust core must be boring*. The frontier of §4.17 may be adopted; the core may not depend on it. Stated as five rules:

1. **Boring cryptography only.** Guarantees G1–G6 rest only on collision-resistant hashing, EUF-CMA signatures, Merkle trees, and ledger persistence — no trusted setup, no SNARK, no oracle, and no token anywhere in the core: nothing about SEAL's integrity, attestation, confidentiality, or existence proofs depends on the price, issuance, or governance of any asset.
2. **Detectability by construction.** Every claim a seal makes is recomputable from public data; there is no shielded region — no analogue of the pool where BCTV14 counterfeits could hide — in which a forged seal can exist unobserved. Forging one requires a hash collision, which is a public event.
3. **The additivity rule.** Frontier components only ever add: zkVM proofs upgrade completeness, restaked capital upgrades confidence, personhood proofs upgrade abuse-resistance — and every one of them can fail to zero, in soundness or in market value, with SEAL degrading exactly to the core guarantees and not one inch below (Figure 7).
4. **The diversity rule.** E12's independent indexer implementations and the substrate's client diversity institutionalize the CVE-2018-17144 lesson: no single codebase is ever the sole witness to the registry's correctness.
5. **Scheduled mortality.** R17 assumes every primitive in use today will eventually fall, and makes renewal a calendar event rather than an emergency (§4.8).

This is the engineering translation of the thesis the user of such a system actually holds: rely on the cryptographic signature — never on a coin.

> **Section takeaways.** The frontier supplies accelerators; history supplies the discipline for adopting them. SEAL takes both: a 2026-current integration surface (§4.17) wrapped around a core that would still verify, unchanged, on the cryptography of 2008.

Part II — Threat Model and Formal Foundations

3 Threat Model, Requirements, and Formal Definitions

A security architecture is only as meaningful as the adversary it names. This section fixes the three pillars on which every later claim rests: a seventeen-capability adversary model that spans byte-level tampering through nation-state coercion and future cryptanalysis (§3.1); twenty engineering requirements R1–R20 that translate that adversary into design obligations (§3.2); and a formal model — an eight-algorithm sealing scheme Σ with six game-based guarantees G1–G6, each tied to a reduction in §6.1 (§3.3). Section 3.4 then converts the evaluation programme into six pre-registered, falsifiable hypotheses with a statistical plan fixed before any data exist. The discipline throughout is explicitness: where a property is guaranteed cryptographically, the game is stated; where it rests on an operational assumption — a quorum, a policy, an indexer — the assumption is named rather than hidden.

3.1 Adversary

The adversary is granted seventeen capabilities, labelled (a)–(q). The first eleven are the classical content-forgery and protocol-level attacks. The adversary can:

- **(a)** modify an artifact's bytes after sealing;
- **(b)** synthesize convincing counterfeits with generative AI;
- **(c)** attempt to backdate — to make content appear older than it is;
- **(d)** repudiate a seal it issued, or falsely claim attestation it never made;
- **(e)** transcode an artifact and claim tampering, or tamper and claim lawful transcoding;
- **(f)** attempt to corrupt the registry at the consensus level;
- **(g)** replay a captured (hash, signature) pair into a different registry, contract, or chain;
- **(h)** seal content it possesses but did not originate — the "valid seal of stolen content" attack;
- **(i)** front-run — observe a published-but-unsealed work, or the mempool, and seal first;
- **(j)** construct adversarial fingerprint preimages — tampered media engineered to remain within distance τ, or benign transcodes pushed outside τ;
- **(k)** mount Sybil and collusion attacks on permissioned validator or polling sets, and exploit ECDSA malleability.

The next six capabilities target the surrounding system rather than the cryptography — the infrastructure, the operators, and the calendar. The adversary can further:

- **(l)** poison or spam the resolution index — inserting fingerprints crafted to saturate the neighbourhoods of popular content, degrading recall and precision for every honest verifier;
- **(m)** operate a malicious verification surface — a lookalike verifier page or application that reports "valid" for forgeries;
- **(n)** control the default mode's L2 sequencer within its finality window — delaying, censoring, or skewing timestamps before L1 inclusion;

- **(o)** record everything now and break today's cryptography later — the harvest-now adversary against signatures and hashes on archival horizons;
- **(p)** compel operators — pinning services, indexers, gateways — through legal or extra-legal pressure;
- **(q)** run dictionary and confirmation attacks against public hashes of low-entropy or confidential content.

What the adversary *cannot* do is equally precise. It cannot break collision-resistant hashing or EUF-CMA-secure signatures during the epoch in which those primitives are current — and the renewal mechanism (§4.8) is precisely the hedge for when this assumption expires. It cannot obtain an honest signer's key. And it cannot achieve the consensus majority of the chosen substrate at the Layer-1/Bitcoin level. Note the deliberately weak shape of this last assumption: honesty is assumed only at L1/Bitcoin — L2-level honesty is never assumed anywhere in the design, which is exactly why adversary (n) appears in the model rather than being defined away.

3.2 Requirements

The adversary model induces twenty requirements. Each one is either discharged by a cryptographic guarantee (§3.3, §6.1) or by a named mechanism in the architecture (§4); none is aspirational.

- **R1 Universality** — seal any file type via a uniform interface.
- **R2 Integrity** — any post-seal byte change is detectable (hard binding).
- **R3 Format robustness (phase 2)** — lawful transcoding does not, by itself, invalidate a media seal; this is explicitly *not* a tamper-detection guarantee within τ against an adaptive adversary (§6.2).
- **R4 Attestation / non-repudiation** — a seal binds to a signer who cannot repudiate it and whom others cannot forge; scoped to attestation-of-possession, not origination or ownership. Multiple independent attestations of the same bytes are legitimate and supported.
- **R5 Existence / anti-backdating** — an upper bound on first-existence time with no trusted timestamping authority, expressed in interval semantics (§4.5) whose adversarial bound is L1/Bitcoin, not the L2 sequencer.
- **R6 Public verifiability** — anyone verifies without privileged access.
- **R7 Lifecycle** — revocation and expiry with enumerated codes, per-code verifier semantics, and an append-only audit trail; revocation withdraws attestation, never the existence proof.
- **R8 Cost and scale** — low per-seal cost at consumer and institutional volumes, including the cost of index maintenance.
- **R9 Privacy** — sealing never publishes artifact bytes; and sealed-but-confidential content must not be confirmable or dictionary-attackable from on-chain data (§4.6).
- **R10 Reproducible verification** — learned fingerprints pin their model (`modelHash`) and archive the model artifact.
- **R11 Anti-replay & domain separation** — the signed payload binds chain-id, registry, contract, and version; replay across registries, chains, or files fails.
- **R12 Key recoverability / rotation** — seals bind to rotatable identities (ERC-1271 / DID controllers) with guardian revocation.
- **R13 Off-chain integrity binding** — the CID is inside the signed commitment; `exactHash` is bound to the bytes the CID resolves to; the CID derivation profile is pinned so recomputation is well-defined.
- **R14 Availability / censorship resistance** — redundant pinning across operators and jurisdictions (adversary p); the on-chain seal is the continuity anchor.

- **R15 Soft-binding resolution** — a verifier can recover a Seal from a stripped or transcoded artifact without an identifier — via an index that anyone can rebuild and whose answers carry proofs (§4.4).
- **R16 Deterministic verification** — every fingerprint is defined by a pinned, deterministic reference implementation (integer-quantized, fixed opset), so recomputation decades later on different hardware is exact.
- **R17 Crypto-agility & renewal** — algorithm identifiers on every hash and signature; epochal re-anchoring such that "anchored before algorithm X broke" is provable forever; optional post-quantum dual-signing.
- **R18 Abuse resistance** — economic backpressure (fees, rate limits) and indexer-side hygiene such that open write access cannot destroy resolution quality; takedown is indexer policy, never on-chain censorship.
- **R19 Finality transparency** — every receipt and verdict carries its anchoring state: pending / L2-final / L1-final / externally anchored.
- **R20 Exportable proofs** — every seal yields a self-contained verification bundle, verifiable offline, surviving the sunset of any L2, indexer, gateway, or vendor.

3.3 Formal model and definitions

Ledger model. The design assumes an append-only ledger satisfying *persistence* — a transaction stably included at height h is never reverted, except with negligible probability — and *liveness* — an honestly submitted transaction is included within a bounded number of blocks — in the sense of the Bitcoin-backbone analysis of Garay, Kiayias and Leonardos [38]. For the default deployment these properties are inherited at the L1/Bitcoin checkpoint level *only*; the L2 sequencer is untrusted within its window (adversary n), which is exactly why every timestamp claim in the system is an interval rather than a point (§4.5). Hash functions are collision-resistant and modeled as random oracles where commitments require hiding; signature schemes are EUF-CMA-secure during their current epoch, with R17 governing the scheduled expiry of that assumption.

The sealing scheme. SEAL is a tuple of eight algorithms:

```
Σ = (Setup, KeyGen, Seal, Anchor, Verify, Resolve, Revoke, Renew)

Setup(1^λ)            → pp        // registry params: domain, suites, index rule, policies
KeyGen(pp)            → (sk, pk)  // or ERC-1271 / DID controller
Seal(sk, F, mode, aux)→ (S, guid) // commitment + signature over F (PUBLIC | CONFIDENTIAL)
Anchor(S)             → ρ         // receipt with interval ⟨t-L2, t-L1, t-BTC⟩ (R19)
Verify(F | guid, B, π_pol) → v    // verdict vector under named policy π_pol
Resolve(F̃, τ, π_pol)  → {(S_i, π_i)} // candidates with inclusion proofs
Revoke(sk_ctrl, guid, c, t) → ε   // coded lifecycle event
Renew(E)              → R_E       // epochal renewal root (§4.8)
```

Here F is a file, $\tilde{F}$ a transformed file, B a verification bundle (§4.9), and $v \in V$ the verdict vector of §4.10. Six guarantees are required of Σ. Each is stated as a game against a probabilistic polynomial-time (PPT) adversary A and proved, by reduction sketch, in §6.1 — the formalization makes precise what a seal does and, just as importantly, what it deliberately does not promise.

G1 (Binding — tamper-evidence). A outputs (F, F', S) with $F \neq F'$ such that `Verify` accepts both under the same seal S with `integrity = exact`. Required: success probability negligible. (→ Theorem 1.)

G2 (Hiding — confidential mode). A chooses Fo, F_1; receives a confidential seal of F_b; outputs b'. Required: $|\Pr[b' = b] - 1/2|$ negligible — the on-chain record leaks nothing about content, defeating the dictionary adversary (q). (→ Theorem 2.)

G3 (Attestation unforgeability). A, given pk and a sealing oracle, outputs a valid seal under pk for a commitment never queried. Required: negligible — this is the EUF-CMA game lifted to the full commitment, domain separation included (R11). (→ Theorem 3.)

G4 (Anti-backdating). A outputs (F, B) such that `Verify` accepts with claimed anchor bound t, yet F embeds a challenger-published fresh random string first released at $t' > t$ — the decidable formalization of "not in A's view". Required: negligible, under ledger persistence at the cited anchoring level — the bound is t-L1/t-BTC, never t-L2 alone. (→ Theorem 4.)

G5 (Resolution soundness). A — a malicious indexer — returns a candidate (S, π) accepted by the verifier although S is not in the anchored log. Required: negligible. Completeness is deliberately *not* a cryptographic guarantee: it rests on the quorum assumption (§4.4, §6.2), and the design says so rather than pretending otherwise. (→ Theorem 5.)

G6 (Renewal preservation). For any seal anchored at epoch Eo and any later epoch E with intact renewal chain, breaking G1/G3/G4 for that seal after the original suite is broken requires breaking a suite that was current at some epoch $\leq E$ *while it was current*. (→ Theorem 6.)

3.4 Research hypotheses, methodology, and statistical plan

The evaluation plan (§9) is restated here as falsifiable hypotheses with a pre-committed statistical methodology, in the registered-report spirit: the analysis is fixed before the data exist, so a failed hypothesis cannot be quietly redefined after the fact.

- **H1 (cost parity → E1, E4).** Anchoring a seal on a blob-era L2 costs ≤ 2× the re-implemented Tio et al. baseline unbatched, and per-seal cost decays as O(1/N) under batching — the measured client-side curve (Appendix E, Figure 8) already exhibits the required shape.
- **H2 (separability → E2).** For ≥ 5 media types, reference fingerprints separate lawful transforms from tampering with AUC ≥ 0.95. Analysis: ROC per modality on ≥ 10,000 labeled pairs; 95% bootstrap confidence intervals; AUC comparisons against PDQ/pHash baselines by DeLong's test; Holm–Bonferroni correction across modalities.
- **H3 (resolution recall → E7).** Recall@1 ≥ 0.9 for stripped/transcoded queries within policy τ, in ≥ 3 modality shards, with Wilson 95% intervals; corpus scaling reported at 10^5, 10^6, and 10^7 sealed items.
- **H4 (abuse economics → E9).** With fees and caps enabled, the attacker spend required to halve Recall@10 on a defended shard is ≥ 10× the undefended cost; reported as a dose–response curve (spend vs recall), not a point estimate.

- **H5 (confidentiality → E10).** Zero confirmations against salted seals at a 2^{40} candidate budget. The executed micro-experiment of Appendix D.5 is the 10^5-scale pilot; Theorem 2 is the asymptotic claim — the two are stated separately precisely so that neither is mistaken for the other.
- **H6 (survivability → E11).** 100% of exported bundles verify offline after staged termination of every live service — L2 RPC, indexers, gateways, pinning.

Datasets and artifacts. Evaluation runs on public, versioned corpora per modality with documented transform and tamper generators; the competitor's image corpus [8] is reused for the head-to-head comparison E8, so the central benchmark cannot be accused of home-field advantage. All code, vectors, and policy profiles ship as citable artifacts — the conformance vector suites are the first instalment.

Ethics and disclosure. Attack tooling for E3/E9/E10 is released responsibly: methods and aggregate curves are published, while weaponizable artifacts are withheld until the corresponding defenses ship. No human-subject data is involved, and sealed test corpora contain no personal documents.

> **Section takeaways.** Twenty requirements (R1–R20), seventeen adversary capabilities (a)–(q), six formal guarantees (G1–G6), and six pre-registered hypotheses (H1–H6) — each guarantee carries a reduction (§6.1), and each hypothesis carries a decision rule (§9). Nothing in the design asks to be believed; everything states how it could be falsified.

Part III — The SEAL Architecture

4 Architecture

Reading map for this section. This is the design chapter, and it presents one structure four times, at four altitudes, each with its own figure. *Statically*, §4.1 (Figure 2) gives the layer stack that every deployment shares — the invariant spine on which everything else is mounted. *Dynamically*, §4.4 (Figure 3) shows how a stripped artifact — one whose identifiers, metadata, and embedded provenance have all been removed — finds its way back to its provenance through the verifiable index. *Temporally*, §4.5–§4.9 (Figures 4–5) show how a seal's meaning is anchored in time, renewed across algorithm generations, and made to survive across decades. And *operationally*, §4.10 (Figure 6) shows how a verifier turns all of it into one reproducible verdict. Sections §4.14–§4.17 then fix the parameters, the contract surface, the scale envelope, and the 2026 integration surface, so that the architecture is buildable from this document alone.

4.1 Layered overview

The architecture is a single spine — identical across every file type and every use case — with optional plugins and a compliance/scale wrapper (Figure 2). The spine reads, bottom to top: **L0 identity & authorship** — keys, ERC-1271/DID controllers, and the identity overlays of §4.11; **L1 format-aware fingerprinting** (§4.2); **L2 timestamp & batching** — Merkle batches and commit-reveal (§5.5); **L3 anchor + verifiable resolution registry** (§4.4–§4.5); **L4 off-chain storage** — IPFS with redundant pinning and optional encryption; **L5 lifecycle** — coded revocation, renewal, and lineage (§4.7–§4.8); and **L7 verification UX** (§4.10). Two layers are plugins rather than spine: **L6** (read-governance and threat detection) and **L8** (multi-party attribution polling). Around the whole sits the wrapper, **L9**: GDPR, the EU AI Act, eIDAS bridging, oracles, monitoring, and APIs. Deployments differ in which plugins and wrapper components they enable; the spine never changes.

One principle organizes the stack, and it is first-class rather than incidental: the architecture is deliberately *hybrid*. Identity and authorization may be centralized per deployment — an institution may gate who writes. Integrity, anchoring, and resolution are decentralized — no operator may gate what is provable. And verification must additionally be *infrastructure-independent*: every guarantee survives the disappearance of any single operator, indexer, gateway, or substrate (R20). The layering is precisely the boundary at which these three trust regimes meet.

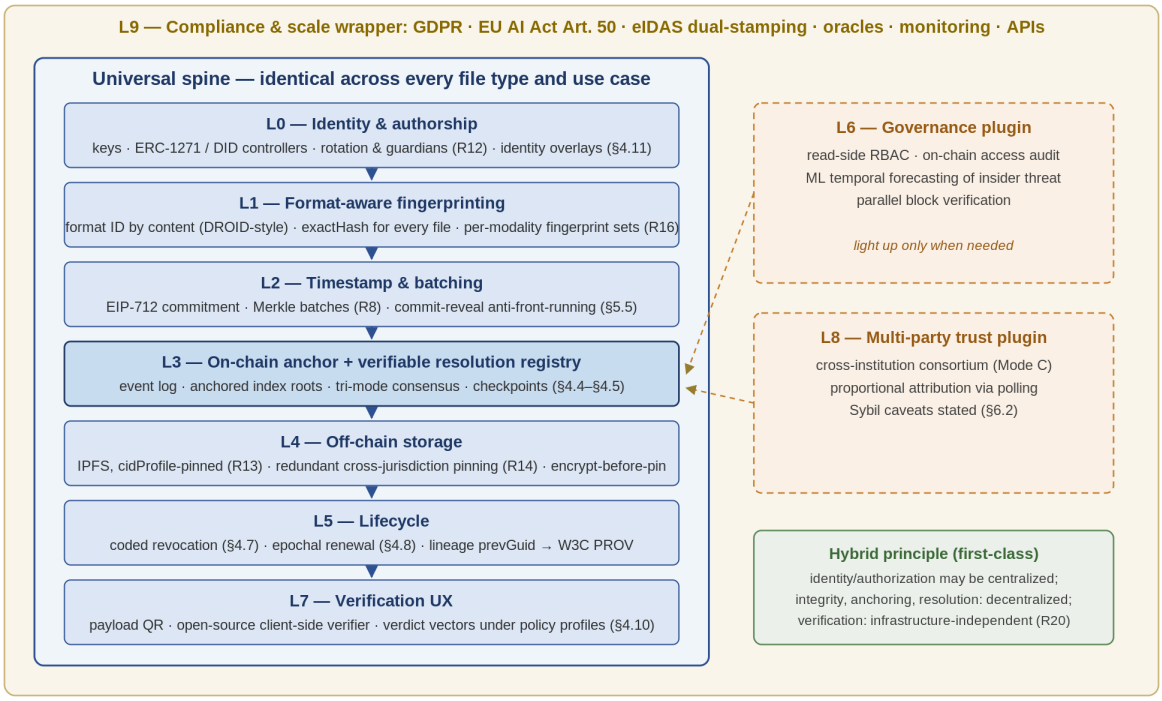


Figure 2. Layered architecture: a universal spine (L0–L5, L7), optional plugins, and the compliance/scale wrapper.

4.2 Format-aware dispatch and the fingerprint set

Following ARCHANGEL's use of DROID, files are classified by binary content — never by extension — and a hashing strategy is dispatched from that classification. The universal baseline, computed for every artifact without exception, is the hard binding `exactHash = H(bytes)` (keccak256 at `exactHashMethodId = 1`). For media and scanned documents, a *fingerprint set* is computed in addition — one or more soft bindings, each declaring its algorithm, its metric, and (for learned methods) its pinned model:

Modality	Fingerprint family	Metric	Notes
Images	PDQ / pHash (256-bit)	Hamming	Phase-2 default; most mature; competitor-compatible
Video	TMK+PDQF	Pooled temporal similarity	Phase 2; frame-level PDQ as fallback shard
Audio	Chromaprint-class	Hamming over binarized chroma	Phase 2
Scanned documents	Learned descriptor (ARCHANGEL-style DNN), quantized	Hamming after binarization	Phase 2; R16 reference implementation mandatory
Text	SimHash / ISCC Content-Code	Hamming	Phase 2; ISCC is ISO 24138
Generic binary	— (exactHash only)	—	Phase 1; R15 not applicable

Two design rules govern this layer, and both follow from refusing a convenient fiction. First, no single `bytes32` + Hamming metric covers "any file type" — the claim does not survive contact with the table above. The Seal therefore carries a fingerprint *set*, Merkle-rooted into the commitment as `fingerprintRoot`, each entry a tuple `(hashMethodId, metricId, fingerprint, modelHash)`; the resolution index shards by `hashMethodId` and searches each shard with the method's own metric. Second, learned fingerprints must sat-

isfy R16 determinism: the normative output is defined by a *reference implementation* — integer-quantized ONNX, pinned opset, CPU execution path — whose artifact is archived and hashed (`modelHash`). Floating-point GPU inference may be used as an accelerator only when it bit-matches the reference; a fingerprint whose value depends on the inference hardware is not evidence. Watermarking (SynthID/Digimarc) remains a complementary in-artifact soft binding, orthogonal to the set carried in the seal. Finally, sealing is not an exclusively human gesture: it may be performed by automated pipelines — ingest daemons, CI steps — wherever bytes cross a custody boundary.

4.3 The canonical Seal record and the signed commitment

A single record represents a sealed artifact, emitted as an event — the log, not contract storage, is the source of truth (§4.4):

```
struct Seal {
  bytes32 guid;              // keccak256(commitment ‖ signer) — signer-specific (R11)
  uint16  exactHashMethodId; // 1 = keccak256; algorithm-agile (R17)
  bytes32 exactHash;         // H(bytes) | H(salt ‖ bytes) in confidential mode (R2; §4.6)
  uint8   sealMode;          // 0 = PUBLIC, 1 = CONFIDENTIAL (§4.6)
  bytes32 fingerprintRoot;   // Merkle root of fingerprint set; 0x0 if none (R3, R15)
  bytes32 chunkRoot;         // root of canonical chunk tree; 0x0 if none (§4.13)
  string  cid;               // content identifier (R13)
  uint8   cidProfile;        // pinned CID version + codec + chunker (R13)
  address issuer;            // organization identity
  string  documentId;        // domain-meaningful id (cert / student / doc no.)
  bytes32 prevGuid;          // lineage edge → PROV wasDerivedFrom; 0x0 if none
  address signer;            // attestation key (R4 — possession, not origination)
  uint16  sigMethodId;       // 1 = ECDSA-secp256k1, 2 = ML-DSA-65, ... (R17)
  bytes   signature;         // over the FULL commitment (R4, R11)
  uint64  declaredTime;      // signer-claimed time — unproven, informational
  uint64  anchorTime;        // block time at inclusion (R5; interval semantics §4.5)
  bytes32 merkleRoot;        // batch root; 0x0 if unbatched (R8)
  string  metadata;          // format id, notes
  uint8   revocationCode;    // 0 = active; codes in §4.7 (R7)
  uint64  revocationTime;
}
```

The signature covers a canonical EIP-712 typed-data commitment with full domain separation:

```
commitment = keccak256(
    EIP712Domain{ name:"SEAL", version, chainId, verifyingContract, registryId }
    ‖ exactHashMethodId ‖ exactHash ‖ sealMode ‖ fingerprintRoot ‖ chunkRoot
    ‖ cid ‖ cidProfile ‖ issuer ‖ documentId ‖ prevGuid ‖ declaredTime ‖ sigMethodId )

signature  = Sign[sigMethodId](commitment)   // ECDSA: low-s canonical form (EIP-2)
guid       = keccak256(commitment ‖ signer)
```

The display abbreviates the standard EIP-712 v4 encoding — `commitment = keccak256(0x1901 ‖ domainSeparator ‖ hashStruct(Seal712))`, `hashStruct = keccak256(typeHash ‖ encodeData)`, with each dynamic field (`cid`, `documentId`) entering `encodeData` as the keccak256 of its UTF-8 bytes. Raw byte concatenation is non-injective over variable-length fields and is therefore forbidden; Appendix D fixes the normative type string.

The commitment binds every integrity-bearing field — fingerprints, the CID *and its derivation profile*, lineage, declared time, and the verifying registry itself — into the attestation, defeating cross-registry and cross-chain

replay (R11). The fields `metadata` and the post-anchor fields are not signed, and verifiers must not render them as attested. Low-s canonicalization kills ECDSA malleability, and the construction makes `guid` deterministic, collision-bound, and signer-specific.

Three behaviors are deliberately *excluded*, and each exclusion is a position. There is no global duplicate-hash guard: any number of signers may seal the same `exactHash`, because attestation is possession, not ownership. Replay protection narrows correspondingly to rejecting duplicate `guid`s. And precedence between rival seals of the same bytes is exactly what §6.2 says it is — a timestamp-ordered evidentiary question for the relying party, never a property the registry adjudicates. Timestamp semantics are likewise split in two: `declaredTime` is the signer's unproven claim; `anchorTime` begins the proven interval (§4.5).

Profile note. The click-pdf production profile (phase-1a/1b) signs a reduced 9-field tuple (`hashMethodId` ≡ `exactHashMethodId`, `assertedAt` ≡ `declaredTime`, `blockTimestamp` ≡ `anchorTime`; an explicit 256-bit nonce; the CID bound as `cidCommit = keccak256(cid)`; `contentHash`/`modelHash` reserved zero), uses the bare EIP-712 digest as its `guid`, a version-"1" domain without `verifyingContract` (version "2" adds it for the anchored profile), and enforces one live anchor per `(exactHash, signer)`; it is not byte-compatible with the v1 encoding of this section.

4.4 The resolution registry, rebuilt: a verifiable index (L3 — core contribution)

The tempting design — placing the similarity index itself on-chain, as Merkle-Patricia-Trie prefix buckets over BK-trees — is not implementable as stated, and it fails on three independent grounds. An index is contract state by definition; every insert mutates O(depth) storage slots; nearest-neighbour traversal does not fit view-function economics. And the escape hatch closes itself: event logs, adopted precisely to avoid state costs, are invisible to contracts — a contract cannot build an index from its own log. The design therefore separates the trust base from the computation, adopting the pattern that Certificate Transparency and Sigstore/Rekor proved at world scale (Figure 3; normative pseudocode in Appendix B):

- **Source of truth — on-chain.** `Sealed` / `Revoked` / `Renewed` events in the registry contract form the append-only log. Writes are cheap because they carry no index state; the log's integrity is the substrate's consensus plus the checkpoint hierarchy (§4.5).
- **Deterministic index rule — specified.** A versioned public specification, pinned by `(indexRuleId, indexRuleHash)` with the same discipline as `modelHash`, maps any log prefix to *exactly one* index state: per-`hashMethodId` shards; MPT prefix buckets + BK-trees for Hamming families (adopted from the 2026 competitor); method-appropriate structures for non-Hamming metrics.
- **Permissionless indexers — off-chain.** Anyone replays the log and rebuilds the index, bit-for-bit. Indexers serve `fetchByContent(fingerprint, methodId, τ, policyId)` → candidate Seals. Running an indexer requires no permission, stake, or identity; institutional verifiers run their own.
- **Anchored index roots — on-chain.** Per epoch, an index checkpoint `(epoch, indexRoot, logRangeCommitment, indexRuleId)` is anchored in the registry. Checkpointing is permissionless and redundant: because the rule is deterministic, all honest checkpointers commit the same root, and any divergence is publicly visible, attributable, and resolvable by recomputation — the dispute is decidable from public data alone (Appendix B.3).
- **Proof-carrying results.** Every candidate returned carries (i) an inclusion proof of the underlying Seal event against the anchored log commitment, and (ii) an inclusion proof of the fingerprint in the anchored `indexRoot`, and (iii) an opening of the returned fingerprint against the candidate seal's `fingerprintRoot` — which lives inside the signed, logged commitment — plus the verifier's own recomputation of

`d(query, fingerprint) ≤ τ`. A verifier checks all of it against the chain — never trusting the indexer or the checkpointer. Anchored roots are claims, not authority (Appendix B.3).

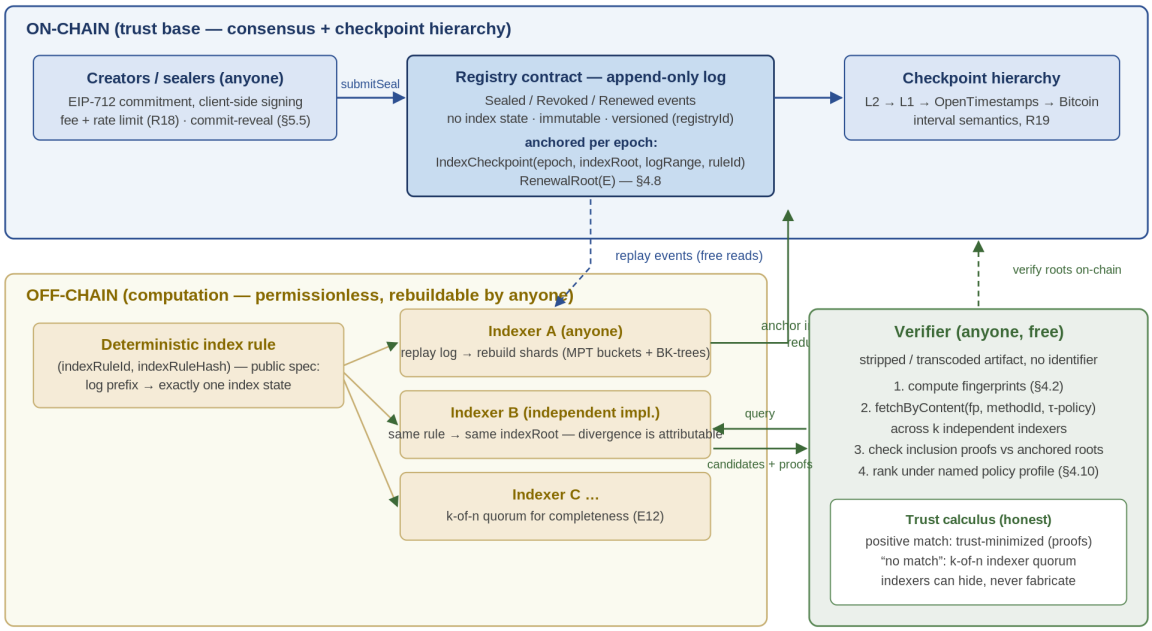


Figure 3. The verifiable resolution index: an on-chain trust base (append-only log + anchored roots), permissionless off-chain computation, and proof-carrying queries. Positive matches are trust-minimized; completeness rests on a k-of-n indexer quorum.

The honest trust calculus. Positive results are trust-minimized: a returned match is either provably in the sealed corpus or rejected (guarantee G5). Completeness — the claim "no seal matches within τ" — cannot yet be proven succinctly for similarity search. The design prices this honestly rather than papering over it: completeness rests on a k-of-n quorum over independent indexer implementations (evaluated in E12), and ZK set-membership/non-membership over the index is named as an open problem (§11). This asymmetry is inherent, not incidental — it is the same asymmetry Certificate Transparency accepts for certificate discovery.

Poisoning and spam defenses (R18). Open writes invite index poisoning: fingerprints crafted to saturate the neighbourhoods of popular content, drowning legitimate matches in noise. Defenses operate at three layers. *Economic* — a per-seal fee and per-signer rate limits make saturation attacks pay-per-byte. *Structural* — per-shard candidate caps and bucket occupancy limits live in the index rule itself, so the deterministic index degrades gracefully and *identically for everyone*; an attacker cannot degrade one verifier's view without degrading their own. *Reputational* — indexers rank candidates by issuer attestations (§4.11), seal age, lifecycle status, and lineage depth, and verifiers see ranking provenance. Abuse of content — illegal-material CIDs — is handled by indexer- and pinning-layer policy (hash-list filtering, takedown propagation), never by on-chain censorship, preserving R6. E9 quantifies recall degradation under calibrated poisoning.

Why this is still decentralized. No operator owns the mapping from artifact to provenance: the rule is public, the inputs are public, anyone can rebuild, every answer is checkable, and the marginal cost of exit — standing up a new indexer — is the cost of replaying the log. "Decentralized" here means *verifiable and operator-independent*, which is strictly stronger than "hosted on a chain."

4.5 Anchoring: tri-mode consensus, the checkpoint hierarchy, and registry governance

Tri-mode consensus. Only the anchoring substrate changes between modes — never the protocol. *Mode A* (default) anchors on a public Ethereum Layer-2 rollup, for public utility. *Mode B* is ARCHANGEL-style public-read / permissioned-write through a sole smart-contract gateway, for controlled issuance. *Mode C* is a permissioned consortium of independently governed institutions — BFT with $n \geq 3f+1$, identity- or stake-gated against Sybil — where corrupting the record requires cross-institutional collusion. In Mode C, archives co-maintain and cross-validate one another's integrity without being party to each other's content until embargo lapses.

The sequencer is in the threat model (R19). On an L2, a centralized sequencer orders transactions and effectively assigns timestamps until the batch lands on L1 — and within that window it can delay, censor, or skew time (adversary n). Timestamp semantics are therefore an *interval*, not a point. `declaredTime` is an unordered, unproven claim — it participates in no inequality. Then: *t-L2* (sequencer-assigned) ≤ *t-L1* (rollup batch finalized on Ethereum) ≤ *t-BTC* (checkpoint aggregated into Bitcoin via OpenTimestamps). Only t-L1 and t-BTC are adversarially meaningful bounds, and receipts and verdicts must state which level they reflect (R19: *pending / L2-final / L1-final / externally anchored*). For consumer UX, L2-final is fine; for court, cite t-L1 or t-BTC. Figure 4(A) summarizes. *Forward note* (§2.7, §4.17): based rollups — L2s sequenced by Ethereum L1 itself, now maturing alongside preconfirmations — would collapse adversary (n)'s window toward L1 inclusion; a based Mode-A variant is the designated path for retiring this concession.

The checkpoint hierarchy. Per epoch, the registry's state commitment — log accumulator, index root, renewal root — is checkpointed: L2 → L1 (inherent to the rollup) → a daily OpenTimestamps aggregation into Bitcoin, at near-zero marginal cost. The hierarchy buys three things. First, an anti-backdating bound independent of the L2's operator. Second, survivability: if the L2 sunsets, every seal remains verifiable from exported bundles (§4.9) against L1/Bitcoin checkpoints. Third, conservatism for century-scale claims, resting the deepest assertions on the most attacked, most replicated chain in existence.

Registry governance. Registry contracts are immutable — no admin keys, no upgrade proxies; an upgradeable registry is a centralized trust root with extra steps. (The phase-1b scaffold in the click-pdf codebase predates this rule — an anchor-only profile with a guardian/pause control and a reduced 9-field commitment; it is not the reference registry of §4.15, and its guardian surface is scheduled for removal before any Mode-A deployment.) Evolution happens by *versioned succession*: a new registry — with a new `registryId`, already bound inside every commitment's domain separator — is deployed permissionlessly; its genesis records a continuity proof, the final checkpoint of its predecessor; discovery routes through social consensus and naming (ENS), not through an authority. Old seals never migrate: they remain verifiable forever in the registry — and in the bundles — that anchored them.

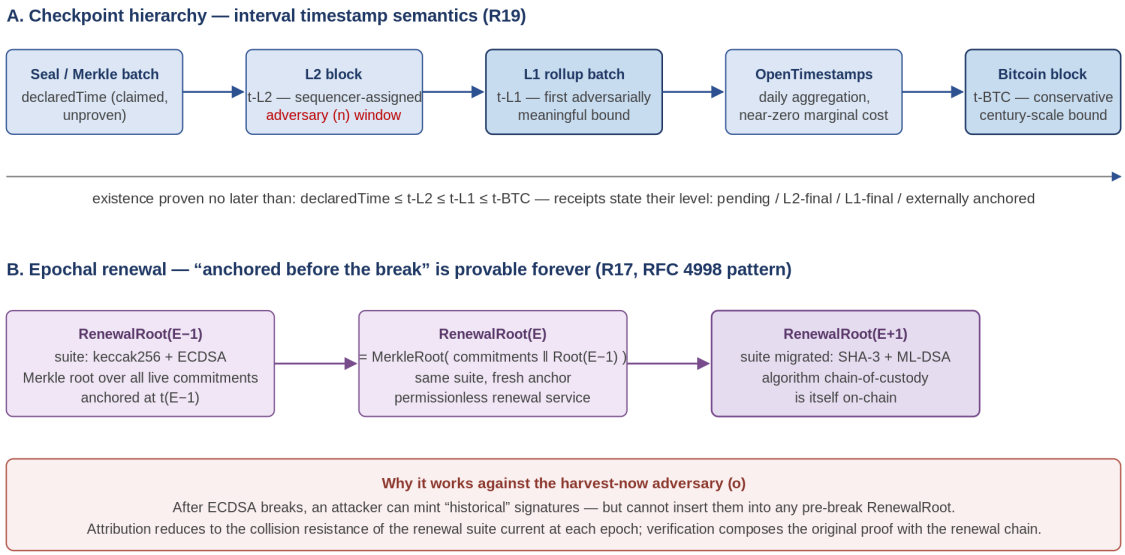


Figure 4. (A) The checkpoint hierarchy and interval timestamp semantics — only t-L1 and t-BTC are adversarially meaningful. (B) Epochal renewal: pre-break anchoring remains provable across algorithm generations (§4.8).

4.6 Confidential sealing, dictionary resistance, and practical erasure

Publishing raw content hashes for every seal would make R9 hollow — the registry itself would become an oracle against the flagship archival use case. Public hashes of low-entropy documents invite dictionary and confirmation attacks: enumerate candidate contents, test each against the hash. Perceptual fingerprints are worse — partially invertible and matchable — so anyone holding a leaked draft could confirm it against a sealed embargoed corpus through the public resolution index. R15 and R9 are in genuine tension, and the resolution is by mode:

- **PUBLIC mode** (default for published content): `exactHash = H(bytes)`; fingerprints indexed immediately; full R15 resolution from day one.
- **CONFIDENTIAL mode** (default for embargoed/institutional content): `exactHash = H(salt ‖ bytes)` with a 256-bit random salt — a hiding *and* binding commitment (guarantee G2), immune to dictionary attacks. Fingerprints are withheld from the public index, or HMAC-keyed for private cross-archive exact-duplicate detection among consortium members — and the scoping here is deliberate: a PRF preserves equality only, never Hamming distance, so HMAC-keyed fingerprints support exact deduplication and nothing more; similarity search over embargoed corpora takes the enclave route of §4.17. At release, the salt (and HMAC key) are disclosed; anyone re-verifies the binding against the original anchor time; fingerprints are computed and only then indexed. The seal proves existence-at-deposition for decades without leaking one bit of content semantics meanwhile.

Consequences. The duplicate-seal question dissolves further: salted commitments of identical bytes differ, consistent with multi-attestation (C2). GDPR: hashes of personal documents can themselves be personal data; in confidential mode, deleting the off-chain bytes *and the salt* renders the on-chain residue unlinkable — practical erasure for the compliance wrapper, with DPIA guidance. Zero-knowledge proofs of seal validity without revealing the artifact remain the natural extension (§11). Public-mode guidance follows directly: unsalted sealing is appropriate only for content that is, or will imminently be, public. Figure 5 traces the full confidential lifecycle — what is public, secret, and provable at deposition, through decades of embargo, and at (possibly partial) release.

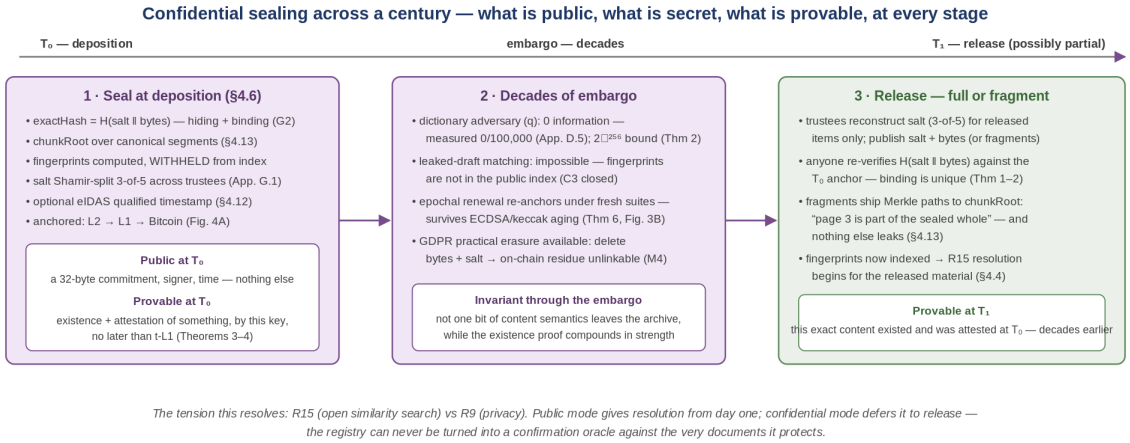


Figure 5. Confidential sealing across a century: a hiding commitment at deposition (T_0), zero leakage through the embargo (measured: Appendix D.5), and provably-authentic full or fragment release at T_1 — resolving the R15-versus-R9 tension by mode.

4.7 Lifecycle: coded revocation, expiry, and lineage

A single `revoked` flag with free-text reasons would collapse cases that demand opposite verifier behavior — disavowal, key compromise, supersession, and clerical error are four different facts with four different consequences. The codes and their semantics are therefore enumerated. Revocation requires authorization from the rotatable controller of `seal.signer` (ERC-1271 wallet / DID controller, optional guardian multi-sig — R12) and emits an append-only `Revoked` event:

Code	Meaning	Verifier semantics
0 ACTIVE	—	Normal verification
1 WITHDRAWN	Signer disavows the content	Attestation void from `revocationTime`; the existence proof stands
2 KEY_COMPROMISE(t)	Signing key compromised at time t	Seals provably anchored before t remain valid; seals after t are void; t asserted by the controller, bounded by the anchor interval
3 SUPERSEDED(guid′)	A successor seal exists	Follow the pointer; treat this seal as historical
4 ERROR	Sealed in error	Attestation void ab initio; the existence proof stands — the mistake is itself evidence

The doctrine is stated once and relied on everywhere: **revocation withdraws attestation, never existence.** A revoked seal still proves that these bytes existed at anchor time; what it no longer carries is the signer's endorsement. Expiry is a declarative metadata field evaluated by policy, not a registry action. Lineage: re-sealing after every curatorial or processing action links versions through `prevGuid` — mapped to W3C PROV `wasDerivedFrom` — so chains of custody are walkable on-chain and checkable in bundles; search-by-guid-or-hash remains a first-class retrieval function.

4.8 Crypto-agility and epochal renewal (R17)

A century-scale thesis cannot stand on 2020s primitives. Algorithm agility is the easy half: every hash and signature in SEAL carries an algorithm identifier (`exactHashMethodId`, `sigMethodId`, and the fingerprint set's `hashMethodId`s), so new algorithms deploy without schema change. The deeper problem is time. When ECDSA eventually falls, an attacker can mint "historical" signatures for any public key — and nothing in the

signature itself distinguishes the genuine 2026 attestation from the forgery computed in 2056. The classical archival answer — RFC 4998 Evidence Record Syntax — is adopted on-chain (Figure 4(B)):

- Per epoch E, a renewal service — permissionless, like checkpointing — collects *all* commitments ever logged, regardless of lifecycle status (revocation never withdraws existence), builds a Merkle tree, and anchors `RenewalRoot(E) = MerkleRoot(commitments ‖ RenewalRoot(E−1))` under the current-best hash and signature suite.
- Verifying an old seal then composes: the original proof path, plus the renewal chain showing the seal was inside `RenewalRoot(E)` anchored at time t(E) — before the break of its original algorithms.
- Forged-after-the-break signatures fail exactly here: they cannot be inserted into a pre-break renewal root. "Anchored before the break" becomes a provable, composable property — indefinitely (guarantee G6).

An archival profile additionally dual-signs commitments with ML-DSA (FIPS 204) alongside ECDSA today. The post-quantum signature lives in the verification bundle (≈ 3.3 kB for ML-DSA-65); the hash of the ML-DSA public key sits *inside the commitment* as a fingerprint-set entry, and is therefore anchored; and the pair `(commitment, H(pqSig))` is bound forward by the next `RenewalRoot` — a signature cannot hash into the very commitment it signs. Harvest-now adversaries (o) gain nothing: confidentiality of sealed content is protected by the salted commitment (§4.6), and attribution is protected by the renewal chain.

Hash migration is stronger medicine, and the distinction matters. Digest-only renewal preserves attestation and anchoring-before-the-break, but it cannot restore *binding* once the old hash admits second preimages — a forged F′ then matches the original `exactHash` inside every renewal root, however many times the digest has been re-anchored. Before a hash suite is deprecated, the byte-holder therefore **re-seals**, binding the new-suite hash of the bytes to the old `guid` via `prevGuid` (RFC 4998 renews by re-hashing *data*, not digests). Post-break verification policies require that new-suite binding for an `integrity = exact` verdict, and in confidential mode only the salt/byte holder can perform the re-hash. The chain of custody of algorithms is itself on-chain.

4.9 The verification bundle (R20)

The unit of long-term survival is not the registry but the bundle. SEAL defines a canonical, self-contained verification file — CBOR-encoded, under a versioned schema whose normative field list appears in Appendix A — that carries everything a verdict requires: the Seal record and its commitment preimage; the signature or signatures, including any post-quantum dual-signature from the archival profile; the Merkle paths that place the seal inside its batch and the batch inside the log; the checkpoint headers at both L2 and L1, together with the OpenTimestamps proof reaching Bitcoin; the renewal-chain paths accumulated to date (§4.8); the snapshot of the policy profile under which the seal was issued; and, for learned fingerprints, the reference-model identifiers — `modelHash` and the CID of the archived model artifact — without which a perceptual verdict could not be reproduced decades later.

The defining property of the bundle is that it verifies offline. Against nothing but widely replicated chain headers, a verifier can recompute the hashes, recover the signer, walk the Merkle paths, and evaluate the policy — with no live L2, no indexer, no gateway, no IPFS node, and no vendor anywhere in the loop. Archives therefore store bundles alongside artifacts as the archival object itself, the format in which a seal outlives every service that produced it. Experiment E11 tests precisely this discipline: kill every service, verify everything anyway.

4.10 Verification UX, the malicious-verifier problem, and policy profiles

The verification surface is itself part of the threat model. Naive QR-and-link verification invites the obvious attack: a forger prints a QR code pointing at their own lookalike verifier, which answers "VALID" for anything presented to it. Adversary (m) is exactly this attacker, and the architecture answers with three structural commitments:

- **QR codes encode a payload, not a URL.** The payload is `(guid, registryId, chainId, schemaVersion)` — the coordinates of a seal, not the address of a service. Any verifier — a wallet, an app, a library — resolves that payload independently against public data. A URL convenience form is permitted, but it must embed the same payload redundantly, so that independent verifiers reach the same verdict on the same input regardless of where the link points.
- **The reference verifier is open-source and client-side.** All verification logic — signature recovery, Merkle and inclusion checks, policy evaluation — runs locally against public chain data. Hosted verifiers exist as conveniences whose answers are reproducible by anyone; they are never authorities.
- **Verdicts are vectors, evaluated under named policies.** A verdict is not a boolean but the tuple `(integrity: exact | failed) × (robustness: within-τ under policy | outside | n/a) × (attestation: valid(signer) | invalid) × (lifecycle: active | revoked(code, t)) × (anchor: pending | L2-final | L1-final | externally anchored)`. The threshold τ for each fingerprint method, and the decision rules that combine the components, come from versioned, citable *verification policy profiles* — the analogue of JPEG Trust's Trust Profiles. Two verifiers operating under the same policy reach the same verdict, and every verdict names the policy it was evaluated under. Threshold governance thereby moves from "whoever wrote the client" to an explicit, auditable artifact.

Figure 6 assembles the full decision flow: both entry paths — identifier-carrying and stripped-file — converging on the five checks in order, and the verdict vector they produce. Every edge in the diagram is a check against public data; no edge is a call to a trusted service.

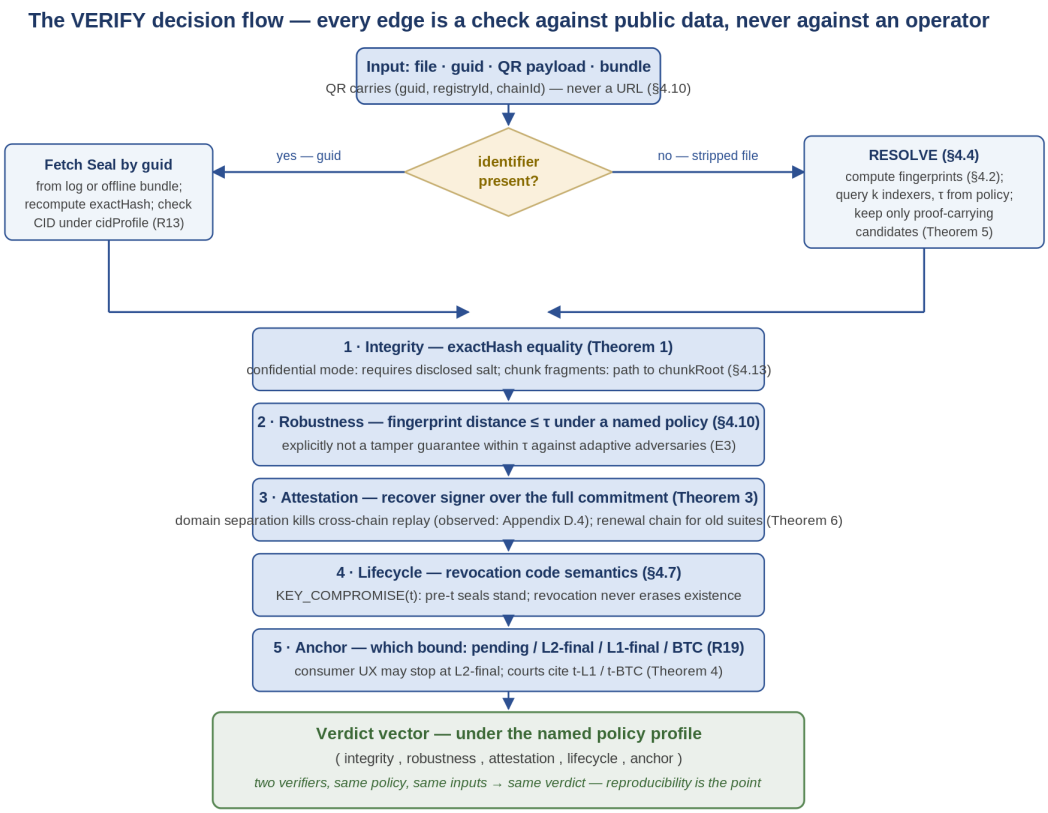


Figure 6. The VERIFY decision flow. Two entry paths converge on five ordered checks — integrity, robustness, attestation, lifecycle, anchor — each grounded in a theorem or a named policy; two verifiers under the same policy reach the same verdict.

4.11 Identity overlays: what replaces the CA

Removing certificate authorities removes their vetting, and the architecture does not pretend otherwise. SEAL's base notion of "who" is deliberately minimal: a key — or a contract wallet or DID with rotation support (R12) — which proves *continuity* of a signing identity, not real-world identity. Where stronger identity is needed, it composes as overlays rather than returning as a root of trust: on-chain attestations in the EAS style, issued by registrars, institutions, or peers, binding a key to an organization or role; W3C Verifiable Credentials and DIDs presented alongside seals; naming systems (ENS) for human-readable continuity; and Mode-B/C deployments in which the write-gateway itself asserts issuer identity — the model of P4. Each overlay is evidence a relying party may weigh; none is a gate the protocol enforces.

The trade-off versus C2PA is stated, not hidden: C2PA gives vetted identity with centralized trust; SEAL gives sovereign keys with composable, inspectable identity evidence. Relying parties choose per policy profile (§4.10) which overlays they require before treating an attestation as identified.

4.12 Standards bridges: C2PA adapter and eIDAS dual-stamping

SEAL does not fork the provenance standards — it decentralizes their weakest joints and connects to the rest. The mapping is one-to-one, and it is worth printing in full:

C2PA / JPEG Trust concept	SEAL element
Hard binding	`exactHash` (+ `cidProfile`-pinned CID)

C2PA / JPEG Trust concept	SEAL element
Soft binding	Fingerprint set entries (`fingerprintRoot`)
Soft Binding Assertion	`(hashMethodId, metricId, modelHash)`
Soft Binding Resolution API	`fetchByContent` with inclusion proofs (§4.4) — exposed behind the C2PA API surface as a decentralized backend
Manifest Store	IPFS-pinned manifest in the bundle; `guid` cross-referenced inside the manifest
Trust List	Identity overlays (§4.11) + verification policy profiles (§4.10)
Claim signature	EIP-712 commitment signature
Trust Profile → Trust Report	Policy profile → verdict vector

The adapter strategy is deliberate adoption economics. A C2PA-compliant pipeline can call SEAL as its Soft Binding Resolution backend without changing any of its manifest tooling; conversely, a SEAL deployment can emit and ingest C2PA manifests. Neither side has to migrate for the systems to interoperate — the decentralized resolution layer slots in behind the existing API surface.

The legal bridge follows the same pattern. An optional eIDAS qualified electronic timestamp over the same commitment is stored in the bundle. Qualified timestamps carry a statutory presumption of accuracy in EU courts that bare chain timestamps lack; dual-stamping combines that presumption with trustless permanence, and aligns with compliance positioning under EU AI Act Art. 50. The framing is kept honest throughout: the seal remains *corroborating* digital-forensic evidence — the DNA-evidence analogy — not a self-executing legal instrument.

4.13 Selective disclosure: chunked commitments

Embargoed inquiries release documents partially — pages, exhibits, fields — and the architecture supports that release pattern natively. The optional chunk tree serves it: the artifact is segmented under a canonical, format-aware chunking specification (pages for PDF; CBOR-path subtrees for structured data; time ranges for audio-visual material), and `chunkRoot` — the Merkle root over the chunk hashes — enters the signed commitment alongside `exactHash`.

One rule is essential to the confidentiality of this mechanism. In CONFIDENTIAL mode the chunk leaves are salted — each leaf is `H(salt_i ‖ chunk_i)` with `salt_i = KDF(masterSalt, i)`, and the per-fragment salts are disclosed only at release time. Without the salts, the chunk tree would itself be a dictionary-confirmable commitment to the embargoed pages, and every fragment proof would expose the sibling page hashes — an attacker holding a guessed page could confirm its presence in the sealed whole. The salting closes both leaks.

Releasing a fragment then ships exactly three things: the fragment, its chunk-hash path to `chunkRoot`, and the seal. The verifier learns that this exact fragment is part of the sealed whole — and nothing else about the rest. Combined with confidential mode (§4.6), an archive can hold a fully secret document for decades and progressively release provably-authentic fragments. The mechanism also aligns with C2PA's redaction assertions, giving the adapter of §4.12 a redaction-compatible path.

4.14 Reference deployment profile and parameterization

The table below fixes normative defaults for the Mode-A reference deployment. Every value is a policy-profile parameter, not a protocol constant; the point of publishing the table is that an implementer can build a complete deployment from it without making any further decisions:

Parameter	Default	Rationale
Hash suite (`exactHashMethodId`)	1 = keccak256; reserved: 2 = SHA3-256, 3 = BLAKE3	EVM-native today; agility via R17
Signature suite (`sigMethodId`)	1 = ECDSA-secp256k1 (low-s); 2 = Ed25519; 3 = ML-DSA-65 (archival dual-sign)	Native `ecrecover`; PQ profile per FIPS 204 (sig ≈ 3.3 kB, in bundle)
Fingerprints (`hashMethodId`)	1 = PDQ-256 (image); 2 = TMK+PDQF (video); 3 = Chromaprint (audio); 4 = quantized scan descriptor; 5 = ISCC/SimHash (text)	§4.2 dispatch table; per-method `metricId`
τ defaults (policy "baseline-v1")	PDQ: 31/256 bits; others calibrated by E2 ROC at FPR $\leq 10^{-3}$	Industry-typical PDQ operating point; policies are versioned and citable (§4.10)
Index rule	`indexRuleId` 1: 16-bit MPT prefix buckets; BK-tree per bucket; bucket cap C = 1024; result cap K = 64	Appendix B; caps bound poisoning damage (R18)
Epoch cadences	Index checkpoint: hourly; OpenTimestamps aggregation: daily; renewal: annually (or on suite deprecation)	Cost amortization vs anti-backdating granularity
Sealing fee + rate limit	Fee ≈ DA cost × small multiple; per-signer N seals/day at base fee, superlinear beyond	Economic backpressure (R18, E9)
On-chain payload per seal	≈ 0.4–0.6 kB event data (unbatched); 32 B amortized root (batched)	§7 worked model
Verification bundle size	≈ 4–10 kB typical; + ≈ 3.3 kB with ML-DSA dual-signature; + model refs for learned fingerprints	Appendix A schema
Storage (R14)	IPFS, `cidProfile` 1 = CIDv1 / raw / 256 kB chunker; ≥ 3 pins across ≥ 2 jurisdictions; Arweave/Filecoin optional	R13 determinism; adversary (p)

4.15 Minimal on-chain interface

The entire contract surface fits on one page — by design. Everything heavy is off-chain and verifiable, and the ABI itself documents the architectural commitment: no index state exists on-chain, and no privileged role exists anywhere in the interface.

```
interface ISealRegistry {
  // ---- append-only log (source of truth) ----
  event Sealed(bytes32 indexed guid, address indexed signer,
               bytes32 indexed exactHash, bytes sealRecord);
  event Revoked(bytes32 indexed guid, uint8 code, uint64 t, bytes32 evidence);
  event IndexCheckpoint(uint64 indexed epoch, bytes32 indexRoot,
               bytes32 logRangeCommitment, uint16 indexRuleId);
  event RenewalRoot(uint64 indexed epoch, bytes32 root,
               uint16 hashMethodId, uint16 sigMethodId);

  function submitSeal(bytes calldata sealRecord, bytes calldata sig) external payable;
  function submitBatch(bytes32 merkleRoot, uint32 count) external payable;
  function commit(bytes32 c) external payable;                  // §5.5
  function reveal(bytes calldata sealRecord, bytes calldata sig, bytes32 nonce) external;
  function revoke(bytes32 guid, uint8 code, uint64 t,
               bytes calldata sealRecord, bytes calldata sig) external;
  function checkpoint(uint64 epoch, bytes32 indexRoot,
               bytes32 logRange, uint16 ruleId) external;       // permissionless
  function renew(uint64 epoch, bytes32 root, uint16 h, uint16 s) external;
  function sealFee() external view returns (uint256);           // R18
}

// No verifyDocument() on-chain: verification is a free off-chain read of the log
// + proofs (§4.4, §4.9). No owner, no upgrade proxy, no admin functions.
```

The interface is not only specified. The reference implementation (`SealRegistry.sol`, in the accompanying reference package) compiles clean and passes a 15-assertion in-EVM suite at 15/15 — including the assertion that closes the loop between paper and code: the contract's `commitmentDigest` and `guidOf` reproduce the client's values byte-for-byte on the same inputs, so a signature accepted by the client encoder is accepted by the chain. Multi-attestation, fee backpressure, replay rejection, malleability rejection, commit-reveal, coded revocation, and permissionless checkpointing are all exercised in code, with measured gas reported in Appendix E.3. Four implementation notes are documented for builders: the batching path emits a dedicated `BatchSealed(merkleRoot, count)` event; `commit()` enforces the fee and rejects re-commits; `revoke()` additionally binds the supplied record to its `guid`; and compilation requires `viaIR`, because the 13-field hashStruct exceeds legacy codegen stack depth.

4.16 Scale and capacity analysis

Capacity is order-of-magnitude engineering, and the architecture is sized at two design points: a national-archive deployment at 10^6 seals and a global public registry at 10^9 seals. The formulas are printed with the figures, so the table regenerates under any future parameters:

Quantity (formula)	10^6 seals	10^9 seals
Raw event log ($\approx b \cdot N$, $b \approx 450$ B)	≈ 0.45 GB	≈ 450 GB
Fingerprint payload (32 B × set size ≈ 2 × N)	≈ 64 MB	≈ 64 GB
Indexer working set (≈ 2–3× fingerprints + structure)	≈ 0.2 GB — laptop-class	≈ 150–200 GB — single commodity server per shard family
Bucket occupancy (N / 2^{16} buckets per shard)	≈ 15 — BK-trees trivially shallow	≈ 15,000 → cap C = 1024 binds per-bucket work; spill chains absorb the tail (Appendix B)

Quantity (formula)	10^6 seals	10^9 seals
Checkpoint chain, 100 years (hourly ≈ 8.77 × 10^5 records ≈ 100 B each)	≈ 88 MB — trivial	identical (independent of N)
Renewal artifacts, 100 years (annual roots + per-seal paths of $\lceil \log_2 N \rceil$ hashes)	20 hashes/seal/century	30 hashes/seal/century — bundle growth ≈ 1 kB/century

Two expert-level caveats belong in print. First, *data availability is not permanence*: post-EIP-4844 blob data expires from consensus nodes after weeks, so long-term availability of the raw log is itself an R14 concern, served by archival replication — indexers, institutions, archive nodes — while verifiability of any individual seal needs only its bundle plus widely replicated chain headers (R20). The design deliberately separates those two survival problems, and assigns them to different actors. Second, BK-tree query cost in high-occupancy buckets degrades toward linear within the bucket; the cap C plus prefix sharding bounds worst-case work per query to $C \cdot B_\tau$ tree visits, where B_τ is the number of buckets intersecting the query ball — exactly the quantity experiment E5 measures against corpus growth.

> **Section takeaways.** Nothing in the architecture has super-linear state anywhere: on-chain cost is O(1) amortized per seal, indexers are commodity hardware even at 10^9 seals, and century-scale checkpoint and renewal overheads are megabytes and kilobytes respectively. The one genuine long-term liability — bulk log availability — is named, separated from verifiability, and assigned to the actors with archival mandates.

4.17 Frontier integrations under the conservative-core filter

Each integration below was selected from the 2026 frontier survey (§2.7) and admitted only if it passes the conservative-core filter (§2.8): it must be *additive* — able to fail to zero, in soundness or in market value, while every guarantee of §6.1 stands unchanged. The third column is therefore the most important one in the table: for every integration it states what the component must never be trusted for, and what happens to the system when it fails. Figure 7 arranges the same content spatially.

Integration (frontier class)	What it adds to SEAL	Conservative-core boundary — what it must never be trusted for
zkVM validity-proved checkpoints (SP1 / RISC Zero class)	Run BuildIndex (Appendix B) inside a zkVM; anchor a validity proof with each `IndexCheckpoint` — build-correctness upgrades from quorum-checked (E12) to proof-carried; a costed path on open problem 1 at 2026 proving prices (§2.7)	Proofs gate checkpoints, never seals. A proving-system soundness bug (the BCTV14 / Frozen Heart class [56][57]) degrades the system back to recomputation-and-quorum — it cannot mint a false seal (Theorem 5 is hash-based)
Restaked indexer / checkpointer set (EigenLayer-class AVS)	Capital-backed completeness confidence: operators bond stake; a divergent index root is an objectively attributable, publicly decidable fault (Appendix B.3) — the cleanest slashing condition available; slashing is live infrastructure since 2025	Economic security buys liveness and honesty incentives, never soundness: a fully slashed — or fully bribed — operator set still cannot fabricate a candidate, because fabrication requires a hash collision, not a majority
PeerDAS-era data availability (Fusaka; DA layers)	Falling anchor costs (blob 14/21 regime; L2 fees −40–60% and trending lower) keep R8 inside its envelope at institutional volume; high-throughput batching for archives	DA expiry is not permanence (§4.16): blob data leaves consensus nodes in weeks; bulk-log availability remains an R14 archival duty; individual verifiability needs only bundles + headers (R20)

Integration (frontier class)	What it adds to SEAL	Conservative-core boundary — what it must never be trusted for
Based sequencing / preconfirmations	A Mode-A variant on a based rollup is sequenced by L1 itself: adversary (n)'s window collapses toward L1 inclusion; preconfirmations give consumer-grade UX receipts without a trusted sequencer	Until adopted, R19 interval semantics stand unchanged; preconfirmations are UX, not anchoring — courts still cite t-L1 / t-BTC
Proof-of-human overlays (World ID 4.0 class; ∼18M verified humans)	Per-human sealing rate limits (R18) without identity disclosure; Sybil-resistant attribution polling for L8; unlinkable per-relying-party proofs match SEAL's privacy posture	Optional, always: personhood never gates verification (R6) or sealing as such — an archive's ingest daemon is not a human and must not need to be
zkTLS / web-origin proofs (DECO lineage [63])	Seal web-native evidence with an origin attestation in the fingerprint set (committed via `fingerprintRoot`): "these bytes were served by origin O at time t" enters the commitment alongside the content hash	Origin ≠ truth and origin ≠ authorship: the attestation scope of R4 is unchanged; a zkTLS break invalidates the origin annotation, never the seal
TEE private matching (interim, until ZK)	Consortium-private similarity search over embargoed corpora: fingerprints decrypted and compared only inside the enclave (HMAC-keyed fingerprints support exact deduplication only — §4.6), serving cross-archive matching without publication	A TEE compromise leaks queries and membership hints — it cannot forge seals or alter verdicts; ZK replaces it on the §11 schedule
C2PA hardware capture (Leica / Sony / Pixel class)	Hardware-attested L0 inputs: capture-time signatures feed identity overlays (§4.11), narrowing the identity gap of §4.11; the sealing pipeline inherits a silicon root of trust it did not have to build	Device-key compromise is bounded exactly like any signer: KEY_COMPROMISE(t) semantics (§4.7); hardware attestation augments, never replaces, the commitment signature
Programmable-IP and provenance chains (Story-class; Numbers-class)	Composition targets: `prevGuid` lineage is a natural substrate for derivative-licensing graphs; C2PA+chain hybrids are interop and migration partners validating the category	Entitlement stays external by design (§6.2): SEAL records possession-at-time; who owns or may license remains the IP layer's claim, never the registry's

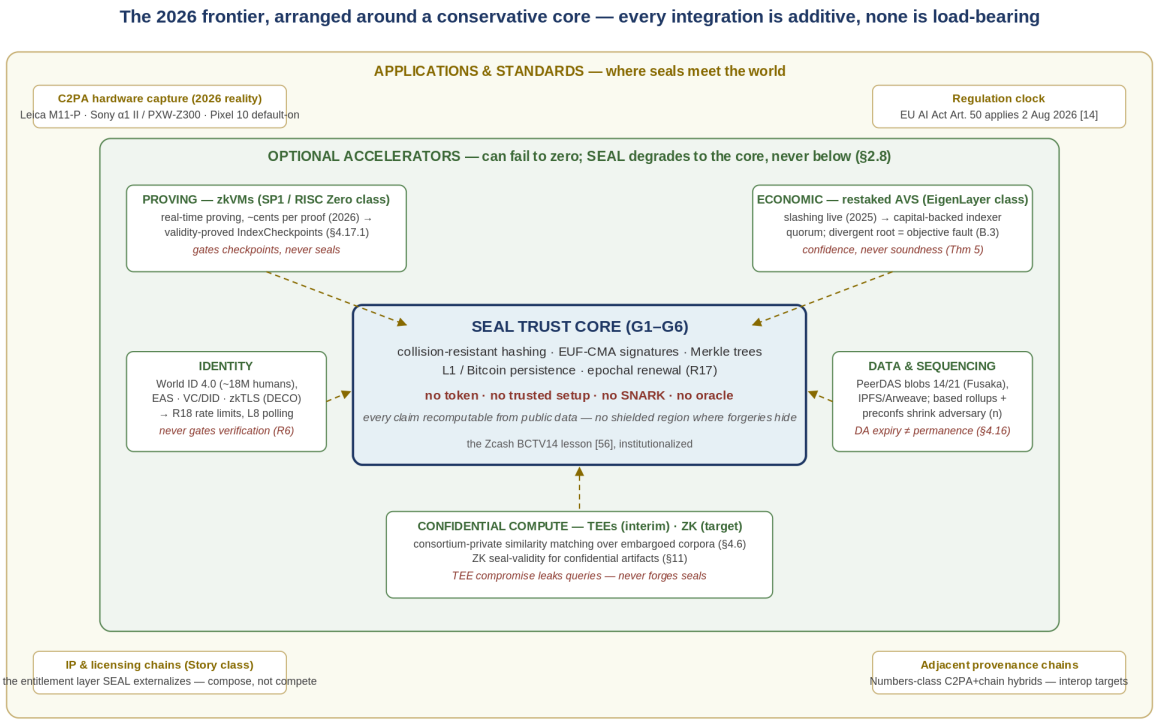


Figure 7. The 2026 frontier arranged around the conservative core: applications and standards outside, optional accelerators in the middle ring — proving, economic, identity, data/sequencing, confidential compute — and a core of plain hashing, signatures, Merkle trees, and persistence. Every arrow points inward and is severable: integrations fail to zero; the core does not move.

> **Section takeaways.** The architecture has a current, costed answer to the 2026 stack — validity-proved checkpoints, restaked quorums, based sequencing, personhood overlays, zkTLS evidence, TEE matching, hardware capture, IP-chain composition — and a single admission rule governing all of it: additive or absent. The frontier accelerates SEAL; it is never permitted to underwrite it.

Part IV — Protocols and Security Analysis

5 Protocol Flows

Six flows constitute the complete operational surface of the architecture: `SEAL` creates a commitment, `VERIFY` checks one, `RESOLVE` recovers one from a stripped or transcoded artifact, `REVOKE` manages its lifecycle, `COMMIT–REVEAL` protects its priority against front-running, and `RENEW` carries the whole corpus across cryptographic generations. Each flow is given as a crisp numbered procedure, and every step names the mechanism of §4 that it invokes, so that the protocol can be audited against the architecture line by line. Nothing in any flow requires permission from an operator; every flow terminates in artifacts a verifier can check independently.

5.1 SEAL(file, sealMode, profile)

Sealing transforms raw bytes into a signed, anchored, exportable commitment. The flow runs client-side until the final submission; no third party sees the plaintext unless the creator chooses a public profile.

1. **Identify the format** (DROID-style, content-based — never extension-trusting); select the fingerprint dispatch and chunking specification for that format per §4.2/§4.13.
2. **Compute the commitment inputs.** Compute `exactHash` — H(bytes) in PUBLIC mode, H(salt ‖ bytes) with a fresh 256-bit salt in CONFIDENTIAL mode. Compute the fingerprint set and the chunk tree where the format admits them; in CONFIDENTIAL mode the fingerprints are withheld from publication.
3. **Pin the bytes** to IPFS under the declared `cidProfile` (encrypting first if confidential); obtain the `cid`.
4. **Sign.** Assemble the EIP-712 commitment (§4.3) and sign it client-side with `sigMethodId`. Under the archival profile, dual-sign with ML-DSA: the post-quantum public-key hash enters the commitment, the ML-DSA signature enters the bundle, and both are bound forward at the next renewal.
5. **Submit.** Optionally enter via commit-reveal (§5.5) when front-running is a concern; otherwise submit directly — `submitSeal(...)` emits the `Sealed` event. Merkle-batch when volume warrants it (R8).
6. **Receive a finality-transparent receipt** (R19) that upgrades through the anchoring ladder — pending → L2-final → L1-final → externally anchored — and export the verification bundle (§4.9).

5.2 VERIFY(file | guid | QR payload)

Verification accepts three entry points — a QR payload, a bare identifier, or a naked file with no identifier at all — and converges on the same verdict machinery in every case.

1. **Decode the entry point.** Decode the QR payload (`guid`, `registryId`, `chainId`, `schemaVersion`) or accept a bare file with no identifier.
2. **With an identifier:** fetch the Seal by `guid` from the log (or from a bundle); recompute `exactHash` under `exactHashMethodId` — in confidential mode this requires the disclosed salt; check the `cid` under `cidProfile`.
3. **Without an identifier:** compute the file's fingerprints; query `fetchByContent(fingerprint, methodId, τ–policy)` across one or more independent indexers; accept only proof-carrying candidates (§4.4).

4. **Check attestation and lifecycle.** Recover the signer over the full commitment (`ecrecover` / ML-DSA verify); evaluate the lifecycle code, the renewal chain (R17), and the anchoring level.
5. **Emit the verdict vector** under the named policy profile (§4.10): integrity / robustness / attestation / lifecycle / anchor. A verdict is never a single bit; each dimension carries its own evidence.

5.3 RESOLVE (stripped or transcoded artifact, no identifier)

Resolution is the adversarially interesting flow: the artifact arrives with no identifier, and the parties answering the query are untrusted. The flow is built so that indexers can only ever *propose*, never decide.

1. **Fingerprint.** Compute fingerprints per modality (§4.2).
2. **Fan out.** Query k independent indexers with (fingerprint, `methodId`, τ from policy); union the candidate sets.
3. **Demand proofs.** Check every candidate's inclusion proofs against the anchored log and index roots; discard unproven answers outright.
4. **Rank and return.** Rank the survivors by policy (issuer overlays, age, lifecycle, lineage); return candidates *with their proofs* — the verifier, not the indexer, decides.

5.4 REVOKE(guid, code, evidence)

Revocation is lifecycle annotation, not deletion: the historical existence proof is untouchable by design, and only the seal's controller can speak about its status.

1. **Authorize** via the rotatable controller of `seal.signer` (ERC-1271 / DID controller; guardian multi-sig supported — R12).
2. **Emit** `Revoked(guid, code, time)` — append-only. `KEY_COMPROMISE` carries its compromise time t; `SUPERSEDED` carries `guid′` (§4.7).
3. **Propagate.** Indexers fold the lifecycle status into ranking within one epoch (R18); the existence proof is untouched by design.

5.5 COMMIT-REVEAL (anti-front-running)

Commit-reveal closes the mempool window during which an observer could copy a pending seal and race it on-chain. It is deliberately a two-step protocol with an honest statement of what it cannot protect.

1. **Commit:** anchor `c = keccak256(commitment ‖ nonce)` — content-blind and unlinkable; the commitment reveals nothing about the artifact, the signer, or even that a seal is coming.
2. **Reveal:** within the policy window Δ, submit the full seal; the registry checks the pair and records both times.
3. **Precedence** runs from commit time. A mempool sniper who copies a revealed seal cannot manufacture an earlier commit. The residual risk — an adversary who saw the work before any commit existed — is unavoidable and stated (priority remains evidentiary, §6.2).

5.6 RENEW (epochal, permissionless)

Renewal is the flow that buys the architecture its archival horizon: it re-binds the entire history under the cryptographic suite of the present, before the suite of the past breaks. Anyone may run it; no seal owner needs to act.

1. **Collect all commitments ever logged at epoch E**; build `RenewalRoot(E) = MerkleRoot(commitments ‖ RenewalRoot(E−1))` under the current suite (§4.8). The recursive inclusion of `RenewalRoot(E−1)` chains every prior epoch into the present one.
2. **Anchor it**; checkpoint to L1 and to OpenTimestamps/Bitcoin (§4.5); distribute updated renewal paths into bundles on demand.

> **Section takeaways.** The six flows close the loop from creation to century-scale survival without a privileged operator anywhere in the path: sealing is client-side until anchoring, verification accepts even a bare file, resolution returns only proof-carrying candidates that the verifier — not the indexer — judges, revocation annotates without ever erasing, commit-reveal makes precedence run from a content-blind commit, and renewal permissionlessly re-binds *all commitments ever logged* under each epoch's current suite.

6 Security Analysis

6.1 Formal guarantees: theorems and proof sketches (G1–G6 of §3.3)

Each guarantee of §3.3 is now discharged as a theorem with a proof sketch, and each reduces to a standard assumption — collision resistance, EUF-CMA security, the random-oracle model, or ledger persistence. The discipline throughout is reductionist: no theorem claims more than its assumption buys, and the places where a claim is deliberately scoped down (L2 granularity in Theorem 4, completeness in Theorem 5, stateful signers in Theorem 3) are stated inside the theorem itself rather than hidden in prose.

Theorem 1 (Binding / tamper-evidence — G1, R2). If H is collision-resistant, no PPT adversary produces $F \neq F'$ both accepted as integrity = exact under one seal, except with negligible probability. *Sketch:* acceptance requires $H(F) = \texttt{exactHash} = H(F')$ (with the same salt in confidential mode); the pair (F, F') is a collision for H. Across signature-suite generations, attestation and anchoring transfer along the renewal chain by Theorem 6; surviving a hash-suite break additionally requires the §4.8 data re-seal. ∎

Theorem 2 (Hiding of confidential seals — G2, R9). Modeling H as a random oracle, a confidential seal with a uniform 256-bit salt gives the dictionary adversary (q) advantage at most $q_H \cdot 2^{-256}$ over guessing, where q_H is its oracle-query budget. *Sketch:* without querying H on (salt ‖ F_b), the commitment is uniform and independent of b; each query succeeds with probability 2^{-256}. Fingerprints are withheld in this mode (`fingerprintRoot = 0x0` or HMAC-keyed), chunk leaves are salted (§4.13), the pinned bytes are encrypted with fresh randomness before the CID is formed, and the game is over equal-length challenges (ciphertext length is otherwise a trivial distinguisher) — closing the auxiliary on-chain leakage channels. Binding follows from collision resistance (Theorem 1). ∎

Theorem 3 (Attestation unforgeability — G3, R4, R11). If the active signature scheme is EUF-CMA-secure, forging a seal attributed to an honest signer reduces to an EUF-CMA forgery. *Sketch:* the commitment — including the EIP-712 domain (`chainId`, `verifyingContract`, `registryId`), all binding fields, and `sigMethodId` — is the signed message; a forged seal on any new commitment is a forgery on a fresh message. Domain separation makes cross-registry/cross-chain replays (g) syntactically distinct messages; low-s canonicalization removes the malleability duplicate. `guid = keccak256(commitment ‖ signer)` prevents cross-signer record replay. The reduction covers key-pair (EOA) suites; ERC-1271 contract signers are stateful, so their seals must carry a state proof in the bundle that `isValidSignature` accepted at the anchor block. ∎

Theorem 4 (Anti-backdating — G4, R5). Under ledger persistence [38] at the cited anchoring level, an artifact whose seal verifies with anchor bound t existed at some time ≤ t, except with negligible probability. *Sketch:* the commitment is a binding function of F (Theorem 1); persistence at L1/Bitcoin implies the commitment's inclusion at the checkpointed height cannot be rewritten; producing the commitment without F requires inverting/colliding H. The claim is deliberately not made at L2 granularity: within the sequencer window (adversary n) only liveness-degraded interval claims hold, which is exactly what R19 receipts expose. ∎

Theorem 5 (Resolution soundness — G5, R15). If H is collision-resistant, no indexer can cause an honest verifier to accept a candidate seal not present in the anchored log. *Sketch:* acceptance requires (i) a Merkle inclusion proof of the `Sealed` event under the anchored log commitment and (ii) inclusion of the fingerprint under the anchored `indexRoot`; fabricating either is a second-preimage/collision on H. Completeness is excluded by construction and handled as a k-of-n quorum assumption (§6.2). ∎

Theorem 6 (Renewal preservation — G6, R17). Let a seal be anchored at epoch Eo and let every epoch boundary since carry a renewal root under the suite current at that epoch. Then violating Theorems 3/4 (attestation, anchoring) for that seal at any later time requires breaking some suite during an epoch in which it was current; binding (Theorem 1) across a hash-suite break additionally requires the pre-deprecation data re-seal of §4.8, which verification policies enforce. *Sketch (ERS argument [30]):* each `RenewalRoot(E)` binds all prior commitments — and `RenewalRoot(E−1)` — under the epoch-E suite before the older suite's break; a post-break forgery must either appear inside a pre-break root (contradicting its collision resistance while current) or fork the anchored renewal chain (contradicting ledger persistence). Hence "anchored before the break" remains provable indefinitely. ∎

Three auxiliary properties fold into the theorems above rather than warranting separate statements. **Off-chain binding** (R13): the `cid` and `cidProfile` live inside the signed commitment, so substituting the pinned bytes breaks Theorem 1 or Theorem 3. **Public verifiability** (R6): all inputs to every verification are public data or bundle contents — no verification step consults a private oracle. **Registry integrity**: rewriting recorded history requires defeating consensus at the checkpoint level, i.e., exactly the persistence assumption of Theorem 4.

6.2 Limitations (stated precisely)

The guarantees above are sharp because their boundaries are. Each limitation below is named with its adversary letter where one exists, its mitigation where one exists, and its residue in every case — the residue is the honest part.

- **Provenance, not truth; attestation, not ownership.** A seal proves an artifact is the one registered, by which key, and when — not that its content is true, the signer honest, or the signer its originator. The "valid seal of stolen content" attack (h) is real and unremoved: anyone possessing bytes can attest possession. Entitlement needs an external claim layer (issuer RBAC, registries); first-to-seal is evidentiary, never dispositive — and multi-attestation makes this honest by construction.
- **Resolution completeness is quorum-trust.** "No match exists" is only as strong as the independence of the k indexers consulted (E12); succinct non-membership proofs for similarity search are an open problem (§11). Positive results carry proofs; absences carry confidence.
- **Perceptual robustness is bounded (j).** R3 is defended against non-adversarial transcodes only; an adaptive adversary can craft tampered preimages within τ (false accept) or push benign transcodes outside τ (false reject). Quantified by E2/E3; policy profiles make the operating point explicit; watermarking complements where it matters.
- **Front-running residue (i).** Commit-reveal protects the window after a creator decides to seal; nothing protects work shown publicly before any commit existed.
- **Poisoning is mitigated, not eliminated (l).** Fees, caps, and ranking raise attack cost and bound damage (E9); a funded adversary still degrades recall within those bounds.
- **Sequencer window (n).** Between submission and L1 finality, ordering and timing are the sequencer's; R19 makes the exposure visible rather than pretending it away.
- **Identity is as strong as its overlays.** A bare key proves continuity only; misvetted overlays mislead exactly as misissued certificates do.
- **Sybil/collusion on permissioned sets (k).** Consortium and polling plugins assume identity/stake-gated membership and $n \geq 3f+1$; a colluding majority remains a residual trust assumption.
- **Availability (R14).** Unpinning loses bytes, not proofs: the seal and bundle survive to verify any extant copy; redundant cross-jurisdiction pinning mitigates compelled takedown (p).
- **No empirical validation here.** This remains a position/architecture paper; §9 commits the falsifiable protocol, including the attacks (first client-side artifacts: Appendices D–E).

6.3 Comparative failure analysis

Security designs are best compared by how they die. Feature matrices flatter every architecture; failure matrices discriminate. The table below traces the five failure modes that matter on archival horizons across the four architectures a relying party can choose today — and reads each cell as the answer to a single question: when this part of the world breaks, what happens to the evidence?

Failure mode	RFC 3161 TSA	C2PA / Trust List	CT-style log	SEAL
Root / authority compromise	TSA key compromise retro-taints all stamps (unless re-stamped)	CA mis-issuance forges identity; trust-list governance is the root	Log key compromise detectable via gossip; split-view possible	No root to compromise: keys are per-signer; `KEY_COMPROMISE(t)` bounds blast radius (§4.7)

Failure mode	RFC 3161 TSA	C2PA / Trust List	CT-style log	SEAL
Operator death	Stamps verify while certs chain; renewal chain dies with operator	Manifest repositories / resolution API go dark; offline manifests survive	Log gone → inclusion proofs unverifiable against future state	Bundles verify offline forever against L1/BTC headers (R20, E11)
Algorithm break	Re-stamping doctrine exists (ERS) but rarely deployed	X.509 migration; old manifests quietly expire	Log hash migration is operator policy	Epochal renewal makes pre-break anchoring provable (Theorem 6)
Legal compulsion / censorship	Operator can be ordered to refuse or backdate-deny	Trust-list removal silently de-authenticates a publisher	Log can be ordered to stop accepting entries	Permissionless writes; takedown only at indexer/pinning policy layer; seals themselves uncensorable (R6, R18)
Lying resolution service	n/a	Centralized Soft Binding Resolution is take-it-or-leave-it	n/a (exact-match only)	Fabrication impossible (Theorem 5); withholding bounded by k-of-n quorum (§4.4)

Read column-wise, the matrix shows three architectures whose failure modes concentrate in an operator, an authority, or a governance body — and one whose failures are bounded per-signer, survivable offline, renewable across suite breaks, and reduced to a quorum assumption precisely where reduction is provably impossible.

> **Section takeaways.** Every guarantee reduces to standard assumptions (collision resistance, EUF-CMA, ledger persistence) — and every residual weakness is named with its blast radius: completeness is quorum-trust, fingerprints are bounded against adaptive adversaries, the sequencer window is exposed rather than hidden. The failure matrix shows the design's differentiation is not features but failure behavior.

Part V — Economics, Pathway, and Evaluation

7 Cost and Scalability

The economics of SEAL are a structural claim, not a price quote. The published baseline of record (P3 [3]) anchors its figures to a mainnet snapshot — 20 gwei, ETH = $3,531.98 — yielding deploy ≈ $84, submit ≈ $6, revoke ≈ $3.5, and free verification, together with its own honesty caveat: the reported latencies were local-Hardhat extrapolations, not public-network measurements. Spot prices of that kind decay the day they are printed; the cost *structure* does not. This paper therefore models the structure. Let `c_DA` be the L2 data-availability price per byte (the post-EIP-4844 blob regime), `c_ex` the L2 execution overhead per transaction, and `b ≈ 450 B` the sealed-event payload (§4.14). The entire on-chain cost surface of the architecture then reduces to five lines:

```
unbatched:  cost(seal) ≈ b · c_DA + c_ex                  // cents-or-below territory
batched(N): cost(seal) ≈ (32 · c_DA + c_ex) / N + b' · c_DA
            where b' ≪ b is the per-item share if leaf data is posted,
            or b' = 0 when leaves live in bundles (existence-only batching)
checkpoint: (epoch cost) / (seals per epoch)  → ~0 marginal
renewal:    (annual root + paths) / corpus     → ~0 marginal
verify:     0 on-chain — free reads + local proof checks
```

Consequences. Five properties follow directly from this structure, and each is either measured or assigned to a named experiment:

- **(i) Batching is asymptotic, not incidental.** Batching drives the anchor cost per seal toward O(1/N): a single 32-byte root amortizes its execution overhead across every leaf beneath it. E4 measures the full curve, including Merkle build time, so the amortization claim is an experimental commitment rather than an asymptotic flourish.
- **(ii) Indexing costs move off-chain by design** (§4.4). Indexers bear replay-and-serve costs — commodity compute, fundable by institutions or fees — and nothing about the index touches gas. The architecture hides no unbudgeted cost anywhere: every expensive operation is either amortized on-chain or explicitly assigned to a permissionless off-chain role.
- **(iii) Checkpointing and renewal amortize over the whole corpus per epoch.** The operations that secure the registry across time — L1 checkpoints, epochal re-anchoring — are per-epoch constants divided by the entire sealed corpus, which is why their marginal cost trends toward zero as the system grows.
- **(iv) Adoption levers are standard, not bespoke.** ERC-4337 account abstraction supersedes the bespoke meta-transaction relayer for gas sponsorship, so users seal without holding ETH; shared organization verifiers amortize deployment cost across an institution rather than charging it to each user.
- **(v) Sustainability is named, not gestured at.** The design replaces "verifier-pays-via-mining" with explicit funding candidates — sealing fees funding an endowment, institutional consortium dues (Mode C), and public-archive statutory budgets — each stress-tested against century-scale obligations in E11's sunset drills.

P7's ≈ 0.0078 ETH/document remains an independent historical cross-check on the unbatched mainnet regime: a figure produced by a different team under different assumptions that lands where the unbatched line

of the model predicts.

Measured. The reference contract puts hard numbers under the model: `submitSeal` = 63,197 gas measured in-EVM — with full EIP-712 verification included in that figure — and a batch root = 23,516 gas, which is 2.4 gas per seal at N = 10,000 (Appendix E.3). At blob-era L2 gas prices these are sub-cent and sub-millicent figures respectively. These are in-EVM measurements of the reference contract, not public-chain results; E1 remains the public-chain confirmation.

7.1 Century-scale economics: the endowment model

Permanent infrastructure needs a funding identity, not a slogan. The model is closed-form. Let `C_annual` be the steady-state operating cost of the public-good components — redundant pinning, at least one always-on public indexer per shard family, checkpoint and renewal gas, and maintenance of the reference verifier — and `r` the real (inflation-adjusted) return of a conservative endowment. Perpetual operation requires `E = C_annual / r`. Illustratively — and explicitly not a forecast — `C_annual ≈ $50k` and `r = 2%` give `E ≈ $2.5M`: a one-time sum well inside a single national archive's digitization budget, fundable by a per-seal fee surcharge at volume (10^9 seals × $0.0025) or by consortium dues (Mode C).

The model's honesty conditions are part of the model. First, `C_annual` is dominated by storage and egress, costs that have fallen for five decades — the endowment is sized against a declining curve, not a rising one. Second, the gas terms amortize per epoch, not per seal (§7), so registry growth does not inflate the obligation. Third, and most important, failure of the endowment degrades *availability*, never *verifiability*: exported bundles keep working (R20). E11's sunset drills are, economically, a test of exactly this separation — they stage the death of every funded service and require the corpus to verify anyway.

> **Section takeaways.** On-chain cost per seal is O(1) amortized and cents-bounded — with 63,197 gas per seal and 2.4 gas per batched seal measured in-EVM; everything heavy is off-chain, commodity, and permissionless; and permanence is priced as a closed-form endowment (`E = C_annual / r`) rather than asserted. The system's economics fail soft: money problems dim the lights, they do not corrupt the record.

8 Implementation Pathway

The architecture deploys in three additive phases, ordered by research risk: the first carries none, the second concentrates all of it, the third extends the horizon. Because the Seal schema reserves every field from day one, later phases activate capabilities rather than migrate formats.

- **Phase 1 — the spine (deployable now).** Format identification; `exactHash`; client-side EIP-712 signing over the full commitment; IPFS pinning with `cidProfile`; the Seal record with `sealMode`, `prevGuid`, and coded revocation; commit-reveal; Merkle batching; checkpointing to L1 plus OpenTimestamps; verification bundles; and the payload QR with an open-source client-side verifier. Everything in this phase is built from standard browser and contract primitives — P3's built-and-benchmarked pipeline and P4's substrate-agnostic prototype are the implementability evidence. No research risk.
- **Phase 2 — fingerprints + verifiable resolution.** Per-modality fingerprint sets under R16 reference implementations; the deterministic index rule (Appendix B); permissionless indexers; anchored index roots; proof-carrying `fetchByContent`; and poisoning defenses. This phase is research-hard, and the hardness is named: model selection, τ calibration, adversarial bounds, and index scaling — exactly the questions E2/E3/E5/E7/E9/E12 are designed to answer before any production claim rests on them.

- **Phase 3 — long-horizon and confidential machinery.** Epochal renewal in production; the ML-DSA archival profile; ZK seal validity for confidential artifacts; ZK (non-)membership for resolution completeness; eIDAS dual-stamp integration; and C2PA adapter certification. Each item extends a guarantee already specified in earlier phases rather than introducing a new trust assumption.

> **Section takeaways.** Phase 1 is deployable now from standard primitives with no research risk; Phase 2 concentrates the genuinely open questions and binds each to a named experiment; Phase 3 adds longevity and confidentiality machinery. The phases are additive — the Seal schema reserves every field from day one, so nothing built in Phase 1 is rebuilt later.

9 Evaluation Plan

The evaluation is a falsifiable protocol, not a benchmark gallery: every requirement receives a named metric, testbed, baseline, and pass/fail criterion — committed in advance, and not yet executed at the protocol level. E1–E8 measure the system; E9–E12 attack it. Latency results report distributions (p50/p95/p99) with disclosed device, browser, network, and trial count, distinguishing optimistic-UI latency from confirmation latency; all adversarial experiments report attack *cost curves*, not just success rates, because a defense that merely raises attacker cost must say by how much. First executed artifacts already exist in the accompanying reference package — conformance vectors, the tested reference contract with measured gas, and client-side microbenchmarks for the Phase-1 spine (Appendices D–E) — and they are explicitly not substitutes for the protocol-level E1–E12.

#	Requirement	Experiment	Baseline	Pass criterion
E1	R2/R8 cost	Deploy factory + Seal to a public L2 testnet; gas/USD for deploy / submit / revoke / verify; with and without blobs	Re-implemented P3, same testbed	$\leq 2\times$ P3; verify = 0
E2	R3 robustness	Fingerprint ROC: (original, lawful-transform) vs (original, tampered); sweep τ; pHash/dHash + learned descriptor, per modality	PDQ / pHash	AUC $\geq$ 0.95 across $\geq$ 5 media types
E3	R3 adversarial	Gradient/black-box preimages within τ; benign transcodes pushed outside τ	—	Report success-vs-cost; no pass threshold (informs τ; "not a tamper guarantee")
E4	R8 batching	Per-seal cost vs batch size N $\in$ {1, 10, 100, 1k, 10k} incl. Merkle build time	Unbatched	Monotone amortization; per-seal cost $\to$ O(1/N)
E5	R8 throughput	Seals/sec under load; pin latency distribution + failure rate; query latency vs corpus size	—	Institutional load sustained; queries sublinear in corpus
E6	R16 determinism	Reference fingerprints across CPU/GPU, frameworks, OSes	—	100% bit-exact under the reference path (conformance, not statistics)
E7	R15 resolution	Recall@k of `fetchByContent` on stripped/transcoded queries over a sealed corpus, per modality shard	Linear scan	Recall@1 $\geq$ 0.9 within τ, $\geq$ 3 modalities

#	Requirement	Experiment	Baseline	Pass criterion
E8	Baselines	Gas/latency/storage vs P3; manifest size + verification time vs C2PA; resolution vs the 2026 competitor on its image corpus	P3, C2PA, [8]	Head-to-head table, p50/p95/p99
E9	R18 poisoning	Calibrated index poisoning at rising spend; measure Recall@k degradation and cost-to-degrade under fees/caps on vs off	Undefended index	Defended cost-to-halve-recall ≥ 10× undefended; degradation bounded by shard caps
E10	R9 dictionary	Confirmation attacks on low-entropy corpora: salted vs unsalted; fingerprint inversion attempts on withheld vs published fingerprints	Unsalted (naive) baseline	0 confirmations against salted mode at 2^40 candidate budget
E11	R20 sunset	Export bundles; kill L2 RPC, indexers, gateways in stages; verify the full corpus offline against L1/BTC headers	No-bundle baseline	100% of seals verify with all services dead
E12	R15 completeness	k independent indexer implementations from the index rule; divergence rate; checkpoint-dispute drill from a seeded faulty root	Single indexer	0 root divergence among honest implementations; faulty root detected and attributed from public data

A red-team phase precedes any production claim: adversaries (g)–(q) are exercised end-to-end against a staging deployment, with findings folded back into the policy profiles and the index rule. The evaluation plan is thus not the last gate before deployment — the adversaries from the threat model are.

> **Section takeaways.** Twelve experiments, six pre-registered hypotheses (§3.4), named baselines, decision rules fixed in advance, attack-cost curves required wherever an adversary appears — and the first instalment (conformance vectors + client-side microbenchmarks) already executed and shipped (Appendices D–E).

Part VI — Context, Law, and Agenda

10 Discussion

A security architecture earns its place not only by its theorems but by how it behaves in its operating context. This section situates SEAL along seven axes: what it discloses and what it withholds (privacy); how it stands before courts and statutes (legal standing); how it integrates into the institutions that will actually run it (the OAIS mapping); how it becomes a standard rather than a product (the standards pathway); how it reframes the deepfake problem; what operational machinery surrounds it; and how it is paid for. It closes — deliberately — with the list of problems the design does not solve.

Privacy. The on-chain footprint is minimal by construction: only commitments, fingerprints (in public mode), CIDs, and metadata ever reach the chain. Confidential artifacts are encrypted before pinning and salted before anchoring (§4.6, Theorem 2), so neither the registry nor any observer of it learns artifact content from the sealing act itself. The natural extension of this posture — zero-knowledge proofs of seal validity that reveal nothing about the artifact at all — is queued as an explicit open problem (§11) rather than claimed as a feature.

Legal standing. Provenance is becoming standardized and regulated — JPEG Trust, EU AI Act Article 50, and the NSA/CISA endorsement of Content Credentials mark the trajectory — and the admissibility map for SEAL is concrete rather than aspirational. *United States:* Federal Rules of Evidence 902(13)–(14) make records "generated by an electronic process shown to produce an accurate result" and "data copied from an electronic device" self-authenticating on a qualified certification [52]. A verification bundle, with its deterministic recomputation procedure, is engineered to be exactly the exhibit such a certification describes: a qualified person can recompute every hash and re-verify every signature and state the result under penalty of perjury. The underlying methods still face Daubert scrutiny [52]; the error characteristics that scrutiny demands are what E2/E3 will publish (ROC curves, adversarial bounds), and the open conformance vectors (Appendix D) are designed to survive precisely this kind of methodological cross-examination. *European Union:* eIDAS grants qualified timestamps a statutory presumption of accuracy — dual-stamping imports that presumption into the seal record (§4.12) — and Art. 46-style non-discrimination clauses bar courts from rejecting electronic evidence solely for its form. *Transferable records:* UNCITRAL's MLETR [53] gives statutory effect to reliable electronic records under a functional-equivalence test whose reliability factors — integrity, identifiability, control — the Seal record answers point by point. The honest framing must survive every retelling: a seal remains *corroborating* digital-forensic evidence — the DNA analogy — never a self-executing instrument. What the engineering buys is that the corroboration is cheap to produce, expensive to fake, and explainable to a finder of fact.

Archival integration: the OAIS mapping. For the institutions that will actually run Mode B/C deployments, SEAL maps onto the OAIS reference model (ISO 14721) [50] without remodeling — the architecture slots into existing archival workflow rather than demanding a new one. Sealing executes at Ingest (SIP → AIP), with the Seal supplying the Fixity component and the lineage chain supplying the Provenance component of the AIP's Preservation Description Information. The verification bundle is packaged inside the AIP itself, so the archive's holdings remain self-verifying even against the registry's death (R20). Epochal renewal (§4.8) is a Preservation Planning activity on the same footing as format migration — scheduled, audited, routine. Dissemination (DIP) ships fragment proofs from the chunk tree, enabling redacted release with intact integrity guarantees (§4.13). And trusted-repository audits (ISO 16363) [50] can cite checkpointed integrity evidence instead of attestation letters — replacing institutional assertion with recomputable fact.

Standards pathway. Three tracks, in increasing ambition. (i) Register SEAL's resolution interface as a conforming C2PA Soft Binding Resolution implementation and the fingerprint identifiers as Soft Binding Assertions (§4.12) — interoperability with the dominant provenance ecosystem from day one. (ii) Submit the verification-bundle schema (Appendix A) and the index rule (Appendix B) for standardization; the natural venues are the IETF (the ERS/RFC-3161 lineage [24][30]) and ISO/IEC JTC 1/SC 29 (the home of JPEG Trust [12]). (iii) Contribute interval-timestamp receipt semantics (R19) to the rollup-standards conversation, where finality transparency is today folklore rather than specification — SEAL's honest intervals are a candidate for making it specification.

Deepfakes. SEAL does not detect deepfakes; it inverts the burden. When sealed provenance becomes cheap, decentralized, and universal, unsealed content becomes suspect — the question shifts from "is this fake?" to "why does this carry no seal?". Watermarking complements the design in-artifact; the resolution registry matches sealed originals against re-encoded derivatives — with proofs (§4.4).

Operational infrastructure. The primitive becomes infrastructure through a surrounding operational layer: oracles bridge external attestations (issuer status, revocation feeds); smart-contract automation triggers on seal/verify/revoke events; the resilience triad maps to redundant pinning, the on-chain continuity anchor, and exported bundles; monitoring and third-party APIs make the guarantee usable as infrastructure rather than as a demonstration; and self-sovereign identity remains the alternative to issuer-asserted identity (§4.11).

Sustainability and incentives. Funding has moved from "verifier-pays-via-mining" to the explicit models of §7. Contribution-weighted attribution polling (the L8 plugin) retains its honest Sybil and power-dynamics caveats — the mechanism is offered as-is, with its failure modes named. Incentive-compatible voting remains future work.

What SEAL deliberately does not solve. Truth of content; authorship and ownership; identity vetting (delegated to overlays); deepfake detection; and similarity-search completeness proofs. Each is named, bounded, and either delegated or queued — not implied away. A system that is explicit about its non-claims is one whose claims can be trusted.

> **Section takeaways.** On-chain disclosure is minimal — commitments, fingerprints (public mode), CIDs, metadata — with encryption before pinning and salting before anchoring for confidential artifacts (§4.6, Theorem 2). The admissibility map is concrete: FRE 902(13)–(14) self-authentication with Daubert error characteristics to be published by E2/E3; eIDAS presumption imported via dual-stamping; MLETR reliability factors answered point by point — the seal always corroborating evidence, never a self-executing instrument. OAIS integration requires no remodeling (Ingest sealing, in-AIP bundles, renewal as Preservation Planning, DIP fragment proofs, ISO 16363 evidence). Standards run on three tracks (C2PA resolution conformance, IETF/ISO schema submissions, rollup receipt semantics). SEAL inverts the deepfake burden rather than detecting fakes, and it names what it does not solve.

11 Conclusion, Open Problems, and the Five-Year Research Agenda

The binding of what/who/when is solved by C2PA and JPEG Trust. What remained open — and what this work delivers in specification form, with stated guarantees — is fourfold. First, the *decentralization of the trust root* with honest interval timestamps (Theorem 4): no single operator's key, clock, or continued existence stands between a seal and its verification. Second, the *resolution of provenance* as a verifiable, rebuild-from-public-data index rather than a hosted service (Theorem 5): the registry is a deterministic function of public inputs, not a

database someone runs. Third, the *unification* of attestation, coded lifecycle, multi-attestation, universality, confidentiality (Theorem 2), and selective disclosure in one primitive — capabilities that today live in separate, mutually unaware systems. Fourth, *longevity engineering* — renewal (Theorem 6), bundles, migration — that lets the guarantee outlive its own infrastructure and its own cryptography. The design now fails visibly where it must fail, and provably where it can succeed.

Five problems remain genuinely open. They are stated as research questions rather than promises:

- **Succinct completeness.** Can "no seal within τ" be proven succinctly — ZK set non-membership over a similarity index, or an authenticated nearest-neighbour structure with verifiable emptiness — at registry scale? This would upgrade resolution completeness from quorum-trust to proof (§4.4). 2026 status note: real-time zkVM proving at cents-per-proof (§2.7) makes validity-proved index *construction* an engineering project (§4.17); succinct completeness for similarity queries — proving the *absence* of neighbours — remains genuinely open and needs proof-friendly index design.
- **Adversarially robust fingerprints.** What are the achievable (FAR, FRR) bounds for perceptual fingerprints under white-box adversaries with bounded perturbation budgets — and are impossibility results lurking where watermarking already found them? E3 measures; theory is open.
- **ZK confidential verification.** Prove integrity + attestation + anchoring of a confidential seal without revealing artifact, salt, or fingerprints — efficiently enough for consumer hardware.
- **Incentive-compatible attribution.** Sybil-resistant, collusion-resistant contribution weighting for the L8 polling plugin, with on-chain budgets.
- **Century-scale economics.** Endowment sizing and governance for permissionless infrastructure carrying statutory archival obligations — a mechanism-design problem, not an engineering one.

The open problems above order themselves into a five-year research agenda with falsifiable exit criteria per phase:

- **Year 1 — spine in production, evaluation executed.** Phase-1 reference implementation on a public L2 testnet → mainnet; E1/E4/E5/E6/E11 executed and published with artifacts; bundle schema and index rule submitted for community review. *Exit:* H1 and H6 decided.
- **Year 2 — resolution at scale.** Phase-2 fingerprint sets and the verifiable index live for images + scanned documents; ≥ 3 independent indexer implementations; E2/E3/E7/E8/E9/E10/E12 executed; first policy profiles ratified. *Exit:* H2–H5 decided; dispute drill performed in public.
- **Year 3 — standards and law.** C2PA adapter certification; IETF/ISO submissions (§10); eIDAS dual-stamp deployments with at least one archive and one newsroom; an FRE 902(13)/(14) admissibility test case prepared with counsel. *Exit:* a seal admitted as evidence, or the doctrinal gap documented.
- **Year 4 — cryptographic depth.** Production ML-DSA dual-signing and first epochal renewal at corpus scale; ZK seal-validity for confidential artifacts in pilot; research results on succinct (non-)membership for similarity search (open problem 1 above). *Exit:* a renewal chain spanning a real suite migration, verified end-to-end.
- **Year 5 — institutional permanence.** Mode-C consortium of ≥ 5 independently governed institutions; endowment capitalized per §7.1; cross-decade governance charter; naming resolved. *Exit:* the system survives its first real operator death with zero verifiability loss — the thesis, demonstrated.

Section takeaways. The contribution, in specification form with stated guarantees: a decentralized trust root with honest interval timestamps (Theorem 4), resolution as a verifiable rebuild-from-public-data index (Theorem 5), one primitive unifying attestation, lifecycle, multi-attestation, universality, confidentiality (Theorem 2), and selective disclosure, and longevity engineering (Theorem 6) that outlives infrastructure and cryptography alike. Five open problems are stated as questions, not promises — succinct completeness, adversarially robust fingerprints, ZK confidential verification, incentive-compatible attribution, century-scale economics — and they order themselves into a five-year agenda whose every phase carries a falsifiable exit criterion, ending with the thesis itself put to the test: operator death with zero verifiability loss.

References and Appendices

References

The architecture synthesizes, and explicitly credits, several distinct literatures: blockchain document-verification systems (P1–P7), the C2PA and JPEG Trust provenance standards, the timestamping and transparency-log lineage from Haber–Stornetta through Certificate Transparency and OpenTimestamps, perceptual-hashing and watermarking research including its impossibility results, post-quantum signature standards, archival science, and the legal instruments under which a seal must ultimately stand as evidence. Citation indices [1]–[65] are stable and are referenced throughout the body text.

1. J. Collomosse et al. ARCHANGEL: Trusted Archives of Digital Public Documents. Surrey / The National Archives / ODI, 2018.
2. O. R. Meher et al. Digital Document Verification System Using Blockchain. IJFMR 6(2), 2024.
3. N. Tio, O. Pribadi, Robet. Secure Document Notarization: A Blockchain-Based Digital Signature Verification System. JIKO 8(3), 2025, 235–243.
4. Tejaswini Ch et al. A Blockchain-Based Secure Document Verification System for Ensuring Data Integrity and Authenticity. IJEDR 14(1), 2026.
5. S. Z. Ansari, S. D. Zade. Blockchain-Enabled Digital Timestamping and Consensus-Based Incentivization for Collaborative Intellectual Property Management. ISI 30(11), 2025, 3077–3082.
6. P. Chinnasamy et al. Blockchain based electronic educational document management with role-based access control using machine learning model. Scientific Reports 15:18828, 2025.
7. K. V. Birhade et al. Enhancing Security and Transparency: The Role of Blockchain in Document Verification. IRJMETS 5(12), 2023.
8. A. Mohit, B. Aggarwal, C. Gondhalekar. Provenance Verification of AI-Generated Images via a Perceptual Hash Registry Anchored on Blockchain. arXiv:2602.02412, 2026.
9. Coalition for Content Provenance and Authenticity. C2PA Technical Specification v2.x; Content Credentials Explainer (spec.c2pa.org), 2024–2025; Content Authenticity Initiative; Microsoft/BBC Project Origin, 2021.
10. C2PA. Durable Content Credentials — Hard Bindings, Soft Bindings, and the Soft Binding Resolution API (Specification 2.1+).
11. Digimarc. C2PA 2.1 — Strengthening Content Credentials with Digital Watermarks, 2024.
12. ISO/IEC 21617-1:2025. Information technology — JPEG Trust — Part 1: Core foundation. ISO/IEC JTC 1/SC 29/WG 1.
13. S. Dathathri et al. Scalable watermarking for identifying large language model outputs (SynthID-Text). Nature 634, 818–823, 2024; Google DeepMind SynthID.
14. European Union. Regulation (EU) 2024/1689 (EU AI Act), Article 50 — transparency obligations for AI-generated/manipulated content.
15. U.S. NSA / CISA. Joint guidance endorsing Content Credentials / C2PA for content provenance.
16. W3C. PROV-DM / PROV-O — The PROV Provenance Data Model.
17. S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008.
18. V. Buterin. Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. 2013.
19. J. Benet. IPFS — Content Addressed, Versioned, P2P File System. arXiv:1407.3561, 2014.
20. R. C. Merkle. A Digital Signature Based on a Conventional Encryption Function. CRYPTO, 1987.
21. W. Burkhard, R. Keller. Some Approaches to Best-Match File Searching (BK-trees). CACM 16(4), 1973.
22. D. Johnson, A. Menezes, S. Vanstone. The Elliptic Curve Digital Signature Algorithm (ECDSA). IJIS 1(1), 2001, 36–63.

23. Ethereum. EIP-712: Typed structured data hashing and signing; EIP-2: low-s signatures; ERC-1271: contract signature validation; EIP-4844: shard blob transactions; ERC-4337: account abstraction.
24. OpenTimestamps. Scalable, trust-minimized timestamping via Bitcoin and Merkle aggregation; IETF RFC 3161: Time-Stamp Protocol.
25. Facebook/Meta. PDQ and TMK+PDQF perceptual hashing; Microsoft PhotoDNA; ISCC — International Standard Content Code (ISO 24138:2024).
26. NIST. FIPS 204 (ML-DSA/Dilithium), FIPS 205 (SLH-DSA/SPHINCS+), 2024; FN-DSA/Falcon — FIPS 206, initial public draft 2025, final pending.
27. B. Laurie, A. Langley, E. Käsper. Certificate Transparency. IETF RFC 6962, 2013 (and RFC 9162, 2021).
28. Sigstore Project. Rekor: a transparency log for software supply-chain signatures (Linux Foundation / OpenSSF).
29. A. Buldas, A. Kroonmaa, R. Laanoja. Keyless Signatures' Infrastructure (KSI): How to Build Global Distributed Hash-Trees. NordSec, 2013 (Guardtime).
30. T. Gondrom, R. Brandner, U. Pordesch. Evidence Record Syntax (ERS). IETF RFC 4998, 2007; RFC 6283 (XMLERS), 2011.
31. European Union. Regulation (EU) 2024/1183 (eIDAS 2.0) — qualified electronic timestamps and the European Digital Identity framework.
32. Ethereum Attestation Service (EAS) — on-chain attestation schemas; W3C Verifiable Credentials Data Model 2.0; W3C DID Core.
33. S. Williams et al. Arweave: the permanent information storage protocol; Protocol Labs, Filecoin.
34. Starling Lab for Data Integrity (Stanford/USC Shoah Foundation). Frameworks for authenticated evidentiary archives.
35. M. Blum. Coin Flipping by Telephone: A Protocol for Solving Impossible Problems. CRYPTO, 1981 (commitment schemes; salted hash commitments as folklore).
36. DROID — Digital Record Object IDentification, The National Archives (UK).
37. C. Bormann, P. Hoffman. Concise Binary Object Representation (CBOR). IETF RFC 8949, 2020.
38. J. Garay, A. Kiayias, N. Leonardos. The Bitcoin Backbone Protocol: Analysis and Applications. EUROCRYPT, 2015.
39. S. Josefsson, I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). IETF RFC 8032, 2017.
40. S. Haber, W. S. Stornetta. How to Time-Stamp a Digital Document. Journal of Cryptology 3(2), 1991, 99–111.
41. D. Bayer, S. Haber, W. S. Stornetta. Improving the Efficiency and Reliability of Digital Time-Stamping. Sequences II, Springer, 1993.
42. A. Buldas, P. Laud, H. Lipmaa, J. Villemson. Time-Stamping with Binary Linking Schemes. CRYPTO, 1998.
43. R. Tamassia. Authenticated Data Structures. ESA, 2003.
44. S. A. Crosby, D. S. Wallach. Efficient Data Structures for Tamper-Evident Logging. USENIX Security, 2009.
45. M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, M. J. Freedman. CONIKS: Bringing Key Transparency to End Users. USENIX Security, 2015.
46. B. Laurie. Certificate Transparency: Public, Verifiable, Append-Only Logs. ACM Queue 12(8), 2014.
47. C. Zauner. Implementation and Benchmarking of Perceptual Image Hash Functions. M.Sc. thesis, Upper Austria University of Applied Sciences, 2010.
48. L. Struppek, D. Hintersdorf, D. Neider, K. Kersting. Learning to Break Deep Perceptual Hashing: The Use Case NeuralHash. ACM FAccT, 2022.
49. H. Zhang et al. Watermarks in the Sand: Impossibility of Strong Watermarking for Generative Models. ICML, 2024 (arXiv:2311.04378).
50. ISO 14721 — Open Archival Information System (OAIS) Reference Model; ISO 16363 — Audit and Certification of Trustworthy Digital Repositories.
51. L. Duranti et al. InterPARES — International Research on Permanent Authentic Records in Electronic Systems; archival diplomatics of digital records.
52. United States Federal Rules of Evidence, Rules 902(13)–(14) (self-authenticating electronic records, eff. 2017); Daubert v. Merrell Dow Pharmaceuticals, 509 U.S. 579 (1993).
53. UNCITRAL. Model Law on Electronic Transferable Records (MLETR), 2017.

54. V. Buterin. Credible Neutrality as a Guiding Principle. Nakamoto.com, January 2020.
55. V. Buterin. Trust Models. vitalik.eth.limo (blog), August 2020.
56. Electric Coin Co. Zcash Counterfeiting Vulnerability Successfully Remediated (BCTV14 soundness flaw; discovered by A. Gabizon, March 2018; disclosed February 2019); CVE-2019-7167.
57. J. Miller (Trail of Bits). Coordinated disclosure of vulnerabilities affecting Girault, Bulletproofs, and PlonK — the "Frozen Heart" Fiat–Shamir implementation flaws, March 2022.
58. Bitcoin Core. CVE-2018-17144 — inflation/double-spend vulnerability, disclosed September 2018.
59. S. Kannan, S. Viswanath et al. EigenLayer: The Restaking Collective (whitepaper, 2023); EigenLayer mainnet slashing (ELIP-002/006), 2025.
60. M. Al-Bassam. LazyLedger: A Distributed Data Availability Ledger (Celestia), arXiv:1905.09274, 2019; Ethereum EIP-7594 (PeerDAS), Fusaka upgrade, December 2025.
61. Succinct Labs. SP1 Hypercube: Real-Time Proving for Ethereum, 2025–2026; RISC Zero zkVM; zkVM proving-cost reporting, 2026.
62. World Foundation. World ID 4.0 — Full-Stack Proof of Human, April 2026 (Orb-verified credentials; unlinkable per-relying-party proofs).
63. F. Zhang, D. Maram, H. Malvai, S. Goldfeder, A. Juels. DECO: Liberating Web Data Using Decentralized Oracles for TLS. ACM CCS, 2020 (zkTLS lineage).
64. Story Protocol. Programmable IP — Proof-of-Creativity and the Programmable IP License (documentation, 2025–2026).
65. Numbers Protocol. C2PA + blockchain provenance with hardware capture (documentation, 2024–2026).

Appendix A — Verification Bundle Schema (normative, v1)

The verification bundle is the archival unit of the architecture: a single, self-contained proof object that carries everything a verifier needs, decades from now, to reach a verdict without consulting any live service. The schema is a CBOR map (RFC 8949) in canonical encoding with deterministic key order, so that two independent implementations serializing the same evidence produce byte-identical bundles. A bundle MUST verify offline given only widely replicated chain headers (R20) — that requirement is what makes the bundle, rather than the registry, the unit that survives infrastructure death.

```
bundle/v1 = {
  1: schemaVersion        // uint = 1
  2: seal                 // the full Seal record (§4.3), CBOR-encoded
  3: commitmentPreimage   // every field of the EIP-712 struct, explicit
  4: signatures           // [ {sigMethodId, sig} ] — ECDSA and optional ML-DSA
  5: batchPath            // Merkle path to merkleRoot (if batched), else null
  6: logInclusion         // path: Sealed event → anchored log commitment
  7: checkpoints          // [ L2 header ref, L1 header ref + receipt proof ]
  8: otsProof             // OpenTimestamps proof to a Bitcoin block header
  9: renewalPaths         // [ {epoch, suiteIds, path → RenewalRoot(E)} ]
 10: lifecycle            // latest Revoked event + inclusion proof, else null
 11: policySnapshot       // policy profile id + hash used at issuance
 12: fingerprintSet       // entries incl. modelHash + model-artifact CID (R10)
 13: chunkSpec            // chunking spec id + chunkRoot openings (if any)
 14: salt                 // ONLY after release of a confidential seal, else absent
 15: extensions           // map — eIDAS QTS token (§4.12), C2PA manifest CID, ...
}
```

Verification order. The verifier evaluates a bundle in a fixed sequence, so that every later check rests on an already-validated foundation: recompute the commitment from the explicit preimage in (3); check each signature in (4) against that recomputed commitment; check (5)→(6) against the anchoring checkpoints in (7);

check the OpenTimestamps proof (8) against a Bitcoin header; walk the renewal paths in (9) oldest-to-newest, enforcing suite currency at each epoch (Theorem 6); apply the lifecycle semantics of (10) per §4.7; and finally evaluate the verdict vector under the policy snapshot in (11). The order is normative: an implementation that signs off on a later step without completing the earlier ones is non-conformant.

Appendix B — Index Rule v1 and the Dispute Procedure (normative sketch)

The resolution index is not a service; it is a deterministic function of public data. This appendix fixes that function — the build rule, the query semantics, and the dispute procedure that makes anchored checkpoints accountable rather than authoritative.

B.1 Build

The index is a deterministic construction over the event log prefix [0, n): any party holding the public log can rebuild it and must arrive at the same `IndexRoot`. Sealed events insert; Revoked events update flags only.

```
BuildIndex(log[0..n)):
  for e in log:                          // Sealed events only; Revoked updates flags
    for fp in e.fingerprintSet:
      shard  = fp.hashMethodId
      bucket = MPT16(fp.bits[0:16])      // 16-bit prefix bucket
      bk     = BKTree(bucket)            // metric = fp.metricId
      if bk.size < C (=1024): bk.insert(fp, e.guid)
      else: spill(fp, e.guid)            // deterministic overflow chain (poisoning cap)
  IndexRoot = MerkleRoot( shards sorted by (shard, bucket) )
```

B.2 Query

`fetchByContent(fp, methodId, τ, policy)`: the resolver visits every bucket whose 16-bit prefix can contain a point within τ of `fp`; performs a BK-tree range search of radius τ within each visited bucket; and collects ≤ K (=64) candidates in deterministic order (distance, then anchorTime, then guid). For each candidate the resolver attaches two proofs — the log-inclusion proof and the index-inclusion proof — and both verify against on-chain anchors (Theorem 5). The verifier therefore never has to trust the resolver's answer set as given: every returned candidate carries its own evidence of membership in both the log and the deterministically built index.

B.3 Dispute

Checkpoints are claims, not authority. On divergence between anchored `IndexCheckpoint(epoch, root, range, rule)` values, any party replays `BuildIndex` over the public log range; the recomputed root identifies the faulty checkpointer(s). Because checkpoints are signed submissions, attribution is on-chain, and honest verifiers simply ignore roots that fail recomputation. No governance action, voting, or trusted referee is required: the dispute is decidable from public data alone (drilled in E12). This is the property that distinguishes the design's resolution layer from a hosted lookup service — disagreement resolves by recomputation, not by appeal.

Appendix C — Notation and Glossary

The paper's notation is fixed here for reference. Each symbol is defined once and used consistently across the body, the theorems, and the appendices.

C.1 Notation

Term / symbol	Meaning
`H`, ‖	Collision-resistant hash (suite per `methodId`); byte-string concatenation
`F`, `F̃`	Original file bytes; a transformed (transcoded/stripped) derivative
`commitment`, `guid`	EIP-712 typed-data hash of all Seal fields (§4.3); keccak256(commitment ‖ signer)
`exactHash` / `fingerprint`	Hard binding (bit-exact); soft binding (perceptual, per-modality), C2PA vocabulary
τ, `metricId`	Similarity threshold under a method's metric, fixed by a named policy profile (§4.10)
`sealMode`, `salt`	PUBLIC | CONFIDENTIAL; 256-bit blinding value making the commitment hiding (G2)
Σ, G1–G6	The scheme (Setup…Renew) and its six guarantees (§3.3), proved as Theorems 1–6 (§6.1)
Anchor interval	⟨declaredTime (unordered claim); t-L2 ≤ t-L1 ≤ t-BTC⟩ — only L1/BTC are adversarial bounds (R19)
`IndexCheckpoint` / `index-RuleId`	Anchored root of the deterministically rebuilt resolution index; the versioned build rule (Appendix B)
`RenewalRoot(E)`	Epoch-E Merkle re-anchoring of all live commitments under the current suite (§4.8)
Verification bundle	Self-contained offline proof object (Appendix A); the archival unit (R20)
Verdict vector	(integrity, robustness, attestation, lifecycle, anchor) under a named policy
P1–P7	Shorthand for the seven synthesized prior systems [1]–[7]

C.2 Acronyms

Acronym	Expansion	Acronym	Expansion
ADS	Authenticated Data Structure	KSI	Keyless Signatures' Infrastructure
C2PA	Coalition for Content Provenance and Authenticity	L1 / L2	Layer-1 / Layer-2 (rollup) blockchain
CBOR	Concise Binary Object Representation	ML-DSA	Module-Lattice Digital Signature Algorithm (FIPS 204)
CID	Content Identifier (IPFS)	MLETR	Model Law on Electronic Transferable Records
CT	Certificate Transparency	MPT	Merkle Patricia Trie
DA	Data Availability	OAIS	Open Archival Information System (ISO 14721)
DID	Decentralized Identifier (W3C)	OTS	OpenTimestamps
DPIA	Data Protection Impact Assessment	PDQ	Perceptual hash algorithm (Meta)

Acronym	Expansion	Acronym	Expansion
EAS	Ethereum Attestation Service	PQ	Post-Quantum
ECDSA	Elliptic Curve Digital Signature Algorithm	PROV	W3C Provenance Data Model
EIP / ERC	Ethereum Improvement Proposal / Request for Comment	QTS	Qualified (electronic) Time Stamp (eIDAS)
ERS	Evidence Record Syntax (RFC 4998)	RBAC	Role-Based Access Control
EUF-CMA	Existential Unforgeability under Chosen-Message Attack	ROC / AUC	Receiver Operating Characteristic / Area Under Curve
FRE	U.S. Federal Rules of Evidence	SSI	Self-Sovereign Identity
GDPR	General Data Protection Regulation	TSA	Time-Stamping Authority (RFC 3161)
IPFS	InterPlanetary File System	VC	Verifiable Credential (W3C)
ISCC	International Standard Content Code (ISO 24138)	ZK	Zero-Knowledge (proof)

Appendix D — Worked Example and Executed Conformance Vectors

This appendix is executable evidence, not illustration. Everything in it was produced by running the Phase-1 reference implementation (`seal_demo.mjs`, shipped in the accompanying reference package together with the machine-readable `SEAL_test_vectors.json`). The values below are **conformance vectors**: any independent implementation of the v1 commitment encoding MUST reproduce every value bit-for-bit. A single deviating byte in any digest, signature, or guid indicates a non-conformant encoder, not an acceptable variation.

Normative encoding notes. The commitment is the EIP-712 typed-data hash of the `Seal` tuple of §4.3, and `registryId` is carried in the EIP-712 domain `salt` field. The demo uses the Sepolia chain-id (11155111), the placeholder verifying contract `0x0000000000000000000000000000000000000001`, and the well-known Hardhat/Anvil development account #1 — published test material, never to be used in production. The EIP-712 domain version string is `"1"`. Dynamic fields (`cid`, `documentId`) enter `encodeData` as the keccak256 of their UTF-8 bytes. ECDSA conformance signing MUST use deterministic RFC 6979 nonces (65-byte `r ‖ s ‖ v` serialization, `v ∈ {27,28}`), so signature vectors are bit-reproducible across implementations and runs.

```
chainId             = 11155111 (Sepolia)
verifyingContract   = 0x0000000000000000000000000000000000000001   // placeholder
domain.version      = "1"
domain.salt         = registryId
test signer         = 0x70997970C51812dc3A010C7d01b50e0d17dc79C8   // Hardhat/Anvil dev account #1
test private key    = 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
                      // published test material — never use in production

commitment struct type string (exact, single line):
Seal712(uint16 exactHashMethodId,bytes32 exactHash,uint8 sealMode,bytes32 fingerprintRoot,bytes32
chunkRoot,string cid,uint8 cidProfile,address issuer,string documentId,bytes32 prevGuid,uint64 de-
claredTime,uint16 sigMethodId)

dynamic fields (cid, documentId): encodeData carries keccak256(UTF-8 bytes)
ECDSA nonces        = deterministic (RFC 6979); 65-byte r ‖ s ‖ v, v ∈ {27, 28}
```

Inputs not listed explicitly in D.1 take the following values throughout: `exactHashMethodId` = 1; `sealMode` = 0 (D.2) / 1 (D.5); `fingerprintRoot` = `chunkRoot` = `prevGuid` = 0x00…00; `cid` = "" (nothing is pinned in the demo); `cidProfile` = 0; `issuer` = `0x0000000000000000000000000000000000000000`.

D.1 Inputs

Document bytes (119 B, UTF-8): *"SEAL reference vector 001 — provenance, not truth: the artifact that was registered, attested by which key, and when."* Signer (test key): `0x70997970C51812dc3A010C7d01b50e0d17dc79C8`. `registryId` = keccak256(`"SEAL/registry/demo/1"`) = `0x655510791a884157610d7097ab54bec0855a43db60494d94a9370936ea3228c3`, carried in the EIP-712 domain `salt` field. `documentId` = `DOC-001`; `declaredTime` = 1750000000; `sigMethodId` = 1 (ECDSA-secp256k1, low-s).

D.2 PUBLIC-mode seal (executed)

```
exactHash        = 0x2d7c873760e246ac8f7810d3cc58c181ff2ff24dd5fe167fee0ee3ace76041dd
commitmentDigest = 0x93eefae82c790bc504581ba21ed72605b04596dbec2d3493d794a26054d563eb
signature        = 0x10257ef1bb3ae6ecaefe3a57cd37e7966519c78dc56e16919dc9b9d865493919
                   6d01a15e94a701ffe7cb38a783736e492523a3f3421534785d7c69cb8cb9a7fc1b
recoveredSigner  = 0x70997970C51812dc3A010C7d01b50e0d17dc79C8   // == signer ✓ (G3)
guid             = keccak256(commitment ‖ signer)
                 = 0xc60261347edb828291e0478e973aea38b5e109ed9139e610bded66d00c4551bd
```

D.3 Tamper-evidence, observed (G1)

Changing one character of the document ("001" → "00l") yields `exactHash` `0xa574a7c9…f15f21` ≠ the sealed value — integrity verdict: **failed**, as required by Theorem 1. The single-character edit is the smallest semantically deceptive change an adversary can make; the commitment detects it unconditionally.

D.4 Cross-chain replay rejection, observed (R11 / Theorem 3)

Verifying the same signature under an otherwise identical domain with `chainId` = 1 recovers `0xFea1Fb5D27809ca28778b3970F1470C12b06fe71` ≠ signer. The replayed seal fails attestation *without any registry-side logic* — domain separation working exactly as proved: the foreign-chain domain perturbs the digest, ECDSA recovery lands on an unrelated address, and the attestation check fails closed.

D.5 CONFIDENTIAL-mode seal and the dictionary experiment (G2 / Theorem 2)

```
salt             = 0x3bda72745501f8387ce518bb053bb6e699d52ae5ee4e491b28c7172ea4ae3e10
exactHash        = keccak256(salt ‖ bytes)
                 = 0x22d00beea3b5c6e16d02147040df1a9988e8a7506335973242ff02624e5722b1
guid             = 0xc66479e7fbbf35007d0bd711c1992c2e5516b3860942cc1a49bbf522a165db48

Dictionary attack, 100,000 candidates of the form "Form 17 — applicant: user{i}":
  vs UNSALTED commitment : target identified at i = 54321  (≈ 0.8 s)   // unsalted = indefensible
  vs SALTED commitment   : 0 matches — trial commitments cannot even be formed
                           without the salt; salt space is 2^256
```

The experiment makes Theorem 2 tangible. Against the unsalted commitment, a single thread enumerates the candidate space and identifies the target document in under a second — low-entropy documents sealed naively are simply readable. Against the salted commitment the attacker cannot even *form* a trial commitment, because every trial requires the 256-bit salt the attacker does not have.

D.6 Determinism (E6-style conformance for the spine)

Independent recomputation of the commitment digest reproduces it exactly — the spine's conformance criterion. R16 applies its stronger form to learned fingerprints in Phase 2; for the deterministic spine, exact reproduction is the only acceptable outcome, and it is the observed one.

D.7 Vector suite v2

The suite expands to **17 executed vectors** (`SEAL_test_vectors_v2.json`, generator `vectors_v2.mjs` in the reference package), pushing the encoding into its corners: *file edge cases* — the empty file, a single zero byte, Hebrew UTF-8 text, and a 100 kB deterministic binary — each sealed in both public and confidential mode with opening checks (the wrong salt is rejected in every case); *multi-attestation* — three signers seal the same bytes and receive three distinct guids (multi-attestation realized in vectors, not just argued); `declaredTime` at its boundary values 0 and $2^{64}-1$; a *chunk-tree fragment proof* in which the genuine page verifies and a forged page fails (§4.13); a *batch inclusion proof*; a *cross-chain replay matrix* in which the signature is accepted only on the signed chain and rejected on mainnet and Polygon ids; and the real ML-DSA-65 dual-signature of E.4. Every invariant the suite asserts holds: confidential openings bind, fragment and batch proofs are sound, and replay is rejected on all foreign chains.

> **Section takeaways.** The v1 encoding is fixed by executed, machine-readable conformance vectors — 17 in suite v2 — that any independent implementation must reproduce bit-for-bit. The vectors demonstrate, by execution rather than assertion: tamper-evidence on a one-character edit (Theorem 1), cross-chain replay failing closed through domain separation alone (Theorem 3), salted confidential commitments surviving a 100,000-candidate dictionary attack that breaks the unsalted variant in $\approx$ 0.8 s (Theorem 2), and deterministic recomputation of the commitment digest.

Appendix E — Reference Microbenchmarks (measured)

Environment: Node v26.0.0, ethers 6.16.0, @noble/post-quantum 0.6.1, single thread, macOS / Intel Core i9-9980HK, pure-JS (no native addons).

Scope caveat, stated plainly. These are client-side micro-measurements of the Phase-1 spine. They are *not* gas costs, *not* L2/L1 confirmation latencies, and *not* the protocol experiments of §9 — E1/E4/E5 on a public testnet remain promised, not executed. What these numbers do establish is narrower and still valuable: the client-side cost of sealing is negligible on commodity hardware, and batch amortization behaves as the O(1/N) criterion of E4 requires.

Operation (client-side)	Measured
Hash + EIP-712 commitment construction	0.80 ms / seal (1,000-iteration mean)

Operation (client-side)	Measured
ECDSA sign (secp256k1, low-s)	1.4 ms / seal (50-iteration mean)
ECDSA recover / verify	5.1 ms / verification (50-iteration mean)
Signature size	65 B (ECDSA); + 3,309 B with ML-DSA-65 dual-signature (constant, FIPS 204)
Verification bundle (Appendix A shape, JSON)	≈ 4.4 kB typical; ≈ 10.9 kB with the hex-encoded ML-DSA dual-signature (JSON; canonical CBOR is smaller — measured by `measure_bundle.mjs`)
Dictionary attack throughput (context for E10)	≈ 50,000 keccak256 trial commitments / s on one thread — confirming why unsalted low-entropy sealing is indefensible and why salted sealing is not (salt space 2^256)

Merkle batching (E4-style, client side). Build time is near-linear (≈ 34 μs per leaf), the per-seal anchor share falls as 32 B / N, and proof length grows logarithmically — bounding bundle growth:

Batch size N	Build time (ms)	Anchor bytes / seal	Proof length (hashes)
1	0.0	32	1
10	0.3	3.2	4
100	3.3	0.32	7
1,000	41.6	0.032	10
10,000	343.8	0.0032	14

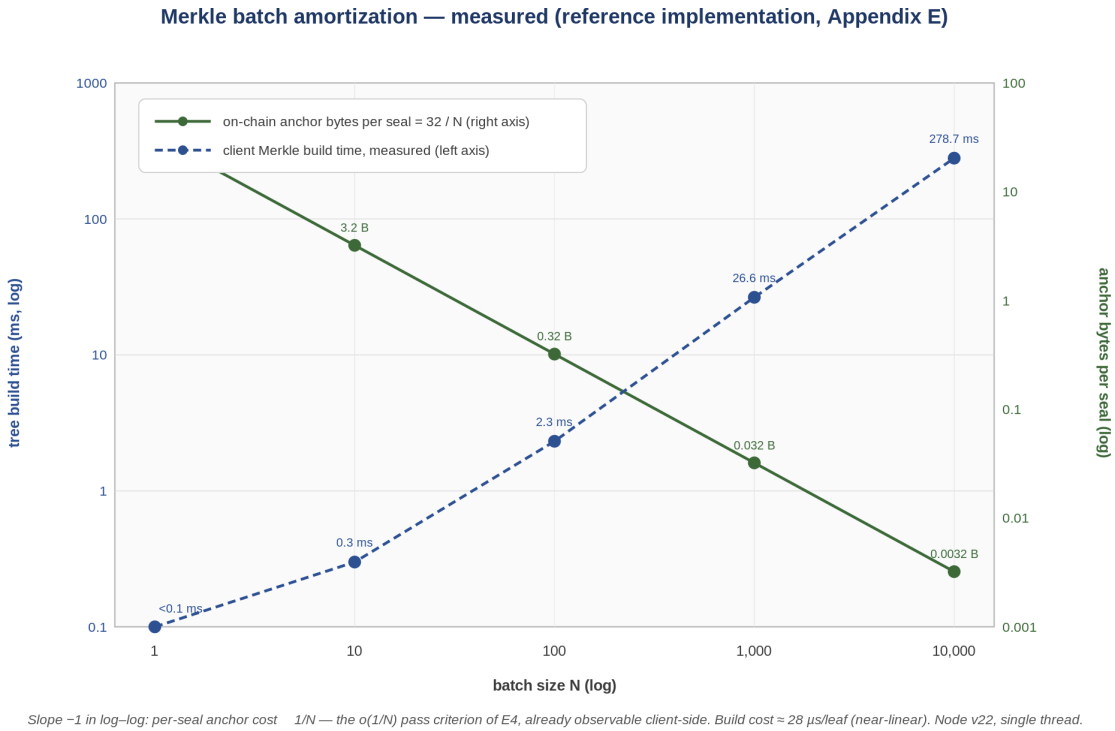


Figure 8. The measured amortization curve (log–log): per-seal anchor bytes fall exactly as 32/N (slope −1) while client build time stays near-linear (≈ 34 μs/leaf) — the O(1/N) shape E4's pass criterion requires, already observable client-side.

E.3 Reference contract: in-EVM measurements

`SealRegistry.sol` (reference package; solc 0.8.35, optimizer 200 runs, viaIR, zero warnings, no admin surface) passes a 15-assertion suite at **15/15** on an in-process EVM (@ethereumjs/vm 10, cancun hardfork, chainId 11155111) — including the conformance assertion that matters most: the contract's `commitmentDigest` and `guidOf` reproduce the client's EIP-712 digests and signer-specific guids *byte-for-byte*. The client and the contract are two independent implementations of the same encoding, and they agree exactly.

Also exercised and passing: duplicate-guid rejection alongside multi-attestation by a second signer (multi-attestation exercised in code, not only in vectors); fee enforcement (R18); cross-registry replay rejection (R11); high-s malleability rejection (EIP-2); commit→reveal with precedence taken from commit time (§5.5); revocation authorization with `KEY_COMPROMISE(t)` storage and an intact existence proof (§4.7); and permissionless checkpointing/renewal driven from an unprivileged account. Measured gas (local EVM — not an L2 measurement; E1 performs the public conversion):

Operation	Gas (measured)	Note
`deploy`	895,102	One-time, factory-amortizable
`submitSeal`	63,197	Full EIP-712 verify + low-s check + event; the headline number
`submitBatch`	23,516	One root for N seals → 2.4 gas/seal at N = 10,000
`commit` / `reveal`	44,355 / 69,316	Anti-front-running pair (§5.5)
`revoke`	42,017	Re-verifies the seal; stores packed (code, t)
`checkpoint` / `renew`	24,712 / 24,396	Permissionless; amortize over the whole corpus per epoch

E.4 Post-quantum dual-signature: measured

The archival profile's ML-DSA-65 dual-signature (§4.8) is no longer a size citation but an executed vector (suite v2, `G-mldsa65-dual-sign`): key generation from a 32-byte seed, signing the real commitment digest, verification passing, and a tampered digest rejected. Measured on pure-JavaScript @noble/post-quantum (hardware implementations are substantially faster): sign ≈ 13.4 ms, verify ≈ 2.4 ms, signature 3,309 B, public key 1,952 B. The conclusion stands with numbers attached: post-quantum dual-signing costs milliseconds and ~3.3 kB per bundle — negligible against archival value.

> **Section takeaways.** Client-side sealing is sub-millisecond to construct and low-single-digit milliseconds to sign and verify; batching amortizes the anchor exactly as 32/N with logarithmic proofs (Figure 8); the reference contract reproduces the client encoding byte-for-byte at 63,197 gas per seal (2.4 gas/seal batched at N = 10,000); and ML-DSA-65 dual-signing adds milliseconds and ~3.3 kB. The honest boundary is unchanged: these are local micro-measurements — the public-testnet protocol experiments (E1/E4/E5) remain promised, not executed.

Appendix F — Anticipated Objections, Answered

The architecture invites a predictable set of challenges. Each is answered here directly, with pointers into the body of the paper where the full argument lives.

"Why not just use C2PA?" We do. SEAL re-implements nothing C2PA solved; it decentralizes the two joints C2PA leaves centralized — the trust root and soft-binding resolution — and exposes itself behind the C2PA Soft

Binding Resolution API as a backend (§4.12). A C2PA pipeline adopts SEAL without changing its tooling.

"Why a blockchain at all? CT, Rekor, and OpenTimestamps exist." Because SEAL needs all three of: permissionless writes (CT/Rekor gate writes through operators), programmable lifecycle (revocation, renewal, fees — logs are passive), and operator-independent survival. Where those systems are strong, SEAL adopts them rather than competing with them: the CT pattern is the resolution layer (§4.4), and OpenTimestamps is the terminal anchor (§4.5). The full argument is §2.3.

"Isn't first-to-seal just ownership by racing?" No ownership claim exists anywhere in the system. Any number of parties may seal the same bytes (multi-attestation, C2); a seal proves possession-at-time by a key, and precedence is evidence courts may weigh — never a right the registry adjudicates (§4.3, §6.2). Commit-reveal (§5.5) removes the mempool race for creators who want it.

"Someone sealed my stolen work." Their seal proves they possessed it at time T — nothing more. Your earlier seal, commit, or any prior evidence outweighs it; their seal even helps you, by fixing the date by which they demonstrably had your work (§6.2).

"Public hashes will leak my confidential documents." Only if sealed naively, unsalted. Confidential mode seals H(salt ‖ bytes): the executed dictionary experiment (Appendix D.5) identifies an unsalted low-entropy document among 100,000 candidates in $\approx$ 0.8 s — and finds nothing against the salted commitment, which Theorem 2 bounds at 2^{-256} per oracle query.

"Quantum computers will break it." Confidentiality is information-theoretically blinded by the salt; attribution survives via epochal renewal — a post-quantum forger cannot insert a signature into a pre-break renewal root (Theorem 6, Fig. 4(B)) — and the archival profile dual-signs with ML-DSA today (§4.8).

"The L2 / the sequencer / IPFS / your indexers will die or cheat." Each is in the threat model (n, p, l). The sequencer is untrusted: only L1/Bitcoin bounds are claimed (R19, Theorem 4). Indexers can hide but never fabricate (Theorem 5), and anyone can rebuild them from public data. Bytes can be lost; proofs cannot: bundles verify offline after every service is dead — drilled in E11 (§4.9).

"Who pays for this over decades?" Anchoring amortizes to ~zero marginal cost per seal (checkpoints and renewal are per-epoch, corpus-wide — §7); indexing is commodity compute open to anyone; and candidate funding models (fee-fed endowment, consortium dues, statutory archive budgets) are stated rather than handwaved, and stress-tested in E11.

"Will a court care about any of this?" A seal is corroborating digital-forensic evidence — deliberately analogous to DNA evidence, not a self-executing instrument. Where statutory weight matters, dual-stamping adds an eIDAS qualified timestamp with its legal presumption (§4.12), and coded revocation semantics keep old evidence interpretable (§4.7).

"What about illegal content?" Bytes never touch the chain — only commitments and identifiers (R9). Filtering and takedown live at the indexer and pinning layers as accountable policy (hash-list filtering, takedown propagation), never as on-chain censorship — preserving both R6 and operator defensibility (R18, §4.4).

"Where's the token?" There isn't one, and that is a design decision, not an omission (§2.8). Nothing in SEAL's integrity, attestation, confidentiality, existence, or resolution-soundness guarantees depends on the price, issuance, or governance of any asset: sealing fees are gas-denominated backpressure (R18), checkpointing and renewal are permissionless services, and restaked capital (§4.17) buys completeness confidence — never soundness. The Zcash BCTV14 episode [56] is the standing reminder of what happens when value security rests on novel cryptography: SEAL's core would still verify on the cryptography of 2008.

> **Section takeaways.** Every anticipated objection resolves into a property the architecture already carries: C2PA compatibility rather than competition, a blockchain only because permissionless writes + programmable lifecycle + operator-independent survival are jointly required, possession-evidence rather than ownership, salted confidentiality with an executed dictionary defense, post-quantum survival by renewal and dual-signing, untrusted intermediaries bounded by theorems, amortized economics, forensic (not self-executing) legal posture, off-chain content policy, and no token by design.

Appendix G — Deployment Case Studies with Failure Analysis

Three deployments, each chosen to stress a different face of the architecture, and each followed by an explicit failure analysis: what breaks, what survives, and what the operator does next. The failure analyses are the point — a provenance system is judged by how it degrades.

G.1 A national archive seals a century-embargoed inquiry

Configuration. Mode C consortium (the archive plus four peer institutions, BFT $5 \geq 3f+1$), CONFIDENTIAL mode, chunk trees over exhibit pages, annual renewal, archival ML-DSA dual-signing, eIDAS dual-stamp at deposition.

Flow. Each deposited document is sealed at ingest (OAIS SIP → AIP; the bundle enters the AIP's Preservation Description Information). Salts are not stored beside the documents — they are Shamir-split 3-of-5 across the consortium trustees, so no single institution can run confirmation attacks against its peers' holdings, and no single loss destroys releasability. At partial release decades later, trustees reconstruct the salt for the released items only, publish fragment proofs from the chunk tree, and the public re-verifies against the original deposition anchor.

Failure analysis. *Institution dissolves:* its checkpoint duties redistribute; verifiability of existing seals is untouched (bundles, R20). *Salt shares destroyed beyond threshold:* the commitment can never be opened — confidential-mode integrity evidence becomes unverifiable; the archive re-seals under fresh salts as a Preservation Planning action, retaining the original anchor as existence corroboration. *Shares leaked beyond threshold:* dictionary-immunity of the unreleased remainder is lost; the remedy is rotation — re-seal plus a fresh Shamir split. *Suite break mid-embargo:* Theorem 6 — the renewal chain proves pre-break anchoring; the inquiry's evidence does not age out with its cryptography. *Court challenge at release:* FRE 902(13)/eIDAS positioning per §10, with the published E2/E3 error characteristics as the Daubert record.

G.2 A newsroom survives platform re-encoding

Configuration. Mode A, PUBLIC mode, fingerprint set {PDQ, TMK} + SynthID-class watermark where the camera pipeline provides it, C2PA manifests emitted in parallel (§4.12), policy profile "newsroom-v1", commit-reveal for embargoed scoops.

Flow. Footage is sealed at ingest by the newsroom's CI pipeline (no human in the loop); the C2PA manifest carries the guid; the story publishes; platforms strip metadata and re-encode. A reader's verifier — or a rival newsroom's — recovers the seal from the stripped derivative via `fetchByContent`, checks the inclusion proofs, and displays the verdict vector with its anchor level.

Failure analysis. *Platform transcode beyond τ:* the multi-fingerprint set degrades gracefully (TMK catches temporal identity when PDQ frames fail); the watermark is the in-artifact fallback; the honest outcome is "no

proven match" — never a false positive (Theorem 5). *Front-runner seals the scoop first:* the newsroom's commit (made at capture time) predates the leak — precedence evidence, §5.5. *Verifier spoofing (adversary m, §4.10):* the printed QR carries the payload, not a URL; any independent verifier reproduces the verdict. *Newsroom key compromised:* `KEY_COMPROMISE(t)` bounds the damage; pre-t footage keeps its attestation.

G.3 A university defends a challenged study

Configuration. Mode A via an institutional gateway (Mode-B style issuer RBAC), PUBLIC mode, seals over: the dataset (by DOI + `exactHash` of the frozen export), the analysis code (`exactHash` per release), and each derived figure (`prevGuid` lineage to data + code).

Flow. At publication, the lab seals the trio; the paper cites guids. Years later, integrity is challenged: any party replays the lineage chain on-chain, fetches the bundles, recomputes hashes over the archived artifacts, and establishes — without trusting the university — that data, code, and figures match deposition exactly (R16 makes the fingerprint recomputation deterministic where media are involved).

Failure analysis. *Lab key lost:* R12 rotation via the institutional controller; historical seals unaffected. *University retracts:* WITHDRAWN revocation — attestation withdrawn, existence proof intact: the record shows both what was claimed and when it was disavowed, which is precisely what research-integrity forensics needs. *Data repository sunsets:* the seal outlives it; any surviving copy of the dataset re-verifies against the bundle (R14/R20 separation, §4.16).

> **Section takeaways.** Across a century-embargoed archive, a metadata-stripping platform pipeline, and a contested research record, the same separations carry the load: existence proofs survive every institutional, key, and storage failure; confidentiality failures are recoverable by rotation and re-sealing; attestation failures are bounded in time by coded revocation; and the honest degradation mode is always "no proven match" or "unverifiable" — never a fabricated positive.

Document status: Version 2.0 — Final Edition, June 2026. A position/architecture paper with formal guarantees and reductions (§3.3, §6.1), a six-decade intellectual lineage (§2.5–§2.6), pre-registered hypotheses and statistical methodology (§3.4), scale and economic models (§4.16, §7.1), comparative failure analysis (§6.3), legal admissibility and OAIS integration (§10), a five-year research agenda (§11), eight figures including a measured amortization curve, and executed artifacts released in the accompanying reference package (`seal_encoding.mjs`, `seal_demo.mjs` + `SEAL_test_vectors.json`, `vectors_v2.mjs` + `SEAL_test_vectors_v2.json`, `SealRegistry.sol` + `run_evm_tests.mjs` + `gas_report.json`, `bench.mjs`, `measure_bundle.mjs`): a reference contract passing 15/15 in-EVM assertions with byte-for-byte client conformance and measured gas, conformance vector suites including real ML-DSA-65 dual-signatures, and client-side microbenchmarks (Appendices D–E). Not yet peer-reviewed or externally audited; the protocol-level evaluation plan (§9) is committed, not executed. The working name SEAL remains provisional.